Universidad Camilo José Cela
SEK EDUCATION GROUP

ADMINISTRACIÓN DE SERVICIOS DE SISTEMAS OPERATIVOS

Código de la asignatura: 88020

Práctica 2
Sistema de archivos, comandos, usuarios y permisos

Integrantes: Guillermo Romo Sánchez
Alonso Malvar Ariza
Ariadna Sánchez Villa

Grupo: G3

Docente: Domingo Sampedro Lirio

Grado: Inteligencia Artificial y Robótica

Curso: 2026-2027 · Octubre de 2026

Índice de contenido

1. Introducción, objetivos y entorno de trabajo....1

1.1. Introducción....1
1.2. Objetivos....1
1.3. Roles del grupo....1
1.4. Entorno y topología....2

2. Primer acceso al sistema e identificación básica....5

2.1. Servidor del grupo....5
2.2. Máquina virtual de Alonso....8
2.3. Máquina virtual de Ariadna....9
2.4. Por qué no se usa root como cuenta habitual....10

3. Creación de la estructura de la empresa....11

4. Usuarios, grupos y roles....15

4.1. Creación de grupos y cuentas....15
4.2. Pertenencia a los grupos de departamento....15
4.3. Comprobación....15

5. Permisos tradicionales (DAC) y configuración 751....19

5.1. Interpretación de 7, 5 y 1....19
5.2. Qué implica el permiso x en un directorio....20

6. Política de acceso por roles y ACLs....21

6.1. Traducción de la política....21
6.2. Criterios de diseño....21
6.3. Cuándo es necesaria una ACL y qué aporta....22
6.4. Configuración realizada por cada integrante....22
6.5. Archivos de ejemplo para las pruebas....26

7. Validación individual de la política de permisos....27

7.1. Pruebas del miembro A: Guillermo (usuario_a)....27
7.2. Pruebas del miembro B: Alonso (alonso)....29
7.3. Pruebas del miembro C: Ariadna (ariadna)....31
7.4. Interpretación....32

8. Incidencias y soluciones....35

9. Conclusiones....37

10. Referencias....39

Anexo A. Script de configuración del servidor....41

Índice de ilustraciones

Ilustración 1. Comprobación de la red y del servicio SSH dentro de la VM con la cuenta guillermo....3
Ilustración 2. Conexión de Ariadna desde su portátil al servidor del grupo con la cuenta ariadna.....3
Ilustración 3. Conexión de Alonso desde su portátil al servidor del grupo con la cuenta alonso.....4
Ilustración 4. Identificación en el servidor con la cuenta de administración guillermo (1/3).....5
Ilustración 5. Jerarquía de primer nivel del servidor con ls -l / (2/3).....6
Ilustración 6. Contenido de /home y acceso a /root sin y con sudo en el servidor (3/3).....6
Ilustración 7. Identificación en el servidor con la cuenta alonso.....7
Ilustración 8. Identificación en el servidor con la cuenta ariadna.....7
Ilustración 9. Identificación en la VM de Alonso con su cuenta de administración al.....8
Ilustración 10. Contenido de /home y acceso a /root en la VM de Alonso.....8
Ilustración 11. Identificación en la VM de Ariadna con su cuenta de administración adminasv.....9
Ilustración 12. Contenido de /home y acceso a /root en la VM de Ariadna.....9
Ilustración 13. Estructura de /srv/aurora6 en el servidor vista desde la cuenta de administración...11
Ilustración 14. Estructura del servidor vista con la cuenta alonso.....12
Ilustración 15. Estructura del servidor vista con la cuenta ariadna.....12
Ilustración 16. Estructura de /srv/aurora6 creada en la VM de Alonso.....13
Ilustración 17. Estructura de /srv/aurora6 creada en la VM de Ariadna.....13
Ilustración 18. Cuentas, grupos y pertenencias comprobadas en el servidor.....16
Ilustración 19. Creación de cuentas y grupos en la VM de Alonso.....16
Ilustración 20. Creación de grupos y cuentas en la VM de Ariadna con cada comando visible.....17
Ilustración 21. Grupos y pertenencias comprobados en la VM de Ariadna.....17
Ilustración 22. Configuración 751 sobre documentacion en el servidor.....19
Ilustración 23. Configuración 751 sobre documentacion en la VM de Alonso.....19
Ilustración 24. Configuración 751 sobre documentacion en la VM de Ariadna.....19
Ilustración 25. ACL de mecanica en el servidor (miembro A).....23
Ilustración 26. ACL de avionica y permisos de software en el servidor (miembro B).....24
Ilustración 27. Política del miembro B aplicada por Alonso en su VM.....24
Ilustración 28. compras, documentacion/comun y ACL de contabilidad en el servidor (miembro C) 25
Ilustración 29. Política del miembro C aplicada por Ariadna en su VM.....25
Ilustración 30. Archivos de ejemplo y resumen de propietarios, permisos y ACL de /srv/aurora6 en el servidor.....26
Ilustración 31. Pruebas de usuario_a en el servidor (1/2).....28
Ilustración 32. Pruebas de usuario_a en el servidor (2/2).....28
Ilustración 33. Pruebas de alonso en el servidor (1/2).....29
Ilustración 34. Pruebas de alonso en el servidor (2/2).....30
Ilustración 35. Pruebas de la política en la VM de Alonso con su cuenta de trabajo alonso.....30
Ilustración 36. Pruebas de ariadna en el servidor (1/2).....31
Ilustración 37. Pruebas de ariadna en el servidor (2/2) y herencia de la ACL por defecto en ensayo_A.txt.....32
Ilustración 38. Pruebas de la política en la VM de Ariadna con su cuenta de trabajo ariadna.....32

Índice de tablas

Tabla 1. Roles del grupo y cuentas de trabajo....1
Tabla 2. Entorno de trabajo del grupo....2
Tabla 3. Comandos de identificación y lo que muestran....5
Tabla 4. Pertenencia de las cuentas de trabajo a los grupos....16
Tabla 5. Descomposición del permiso 751 sobre documentacion....19
Tabla 6. Matriz de accesos derivada del enunciado....21
Tabla 7. Propietarios, permisos y ACL finales....21
Tabla 8. Pruebas de usuario_a en el servidor....27
Tabla 9. Pruebas de alonso....29
Tabla 10. Pruebas de ariadna....31
Tabla 11. Incidencias y soluciones....35

1. Introducción, objetivos y entorno de trabajo

1.1. Introducción

En esta práctica hemos administrado desde la terminal el Ubuntu Server instalado en la P1 para preparar la infraestructura de la empresa ficticia AURORA Aerospace (proyecto AURORA-6): una estructura de directorios en `/srv/aurora6`, una cuenta Linux por integrante, los grupos de cada departamento y una política de acceso que combina permisos tradicionales (DAC) con listas de control de acceso (ACL). La misma infraestructura se reutilizará en las prácticas P3, P4 y P5, así que hemos documentado cada decisión de forma que se pueda reproducir.

Hemos organizado el trabajo como un sistema real compartido. El ordenador de Guillermo actúa como servidor del grupo (ejecuta la máquina virtual `server-asio-g3`) y los portátiles de Alonso y Ariadna actúan como clientes, desde los que cada uno inicia sesión por SSH con su propia cuenta. Además, Alonso y Ariadna han repetido la práctica completa en sus propias máquinas virtuales de la P1, de modo que cada integrante aporta evidencias con su identidad, como pide la regla de trabajo individual del enunciado.

1.2. Objetivos

- Identificar el usuario, el nombre del sistema y la jerarquía básica de directorios del servidor.
- Crear desde la terminal la estructura de directorios de la empresa en `/srv/aurora6`.
- Crear las cuentas de los tres integrantes, los grupos de departamento y el grupo común `proyecto`.
- Interpretar los permisos tradicionales a partir de la configuración 751.
- Traducir la política de roles de la empresa a propietarios, grupos, permisos DAC y ACL, justificando cuándo hace falta una ACL.
- Validar la política con pruebas reales de cada usuario, tanto de operaciones permitidas como denegadas.

1.3. Roles del grupo

Tabla 1. Roles del grupo y cuentas de trabajo

Miembro	Integrante	Cuenta	Rol	Grupos
A	Guillermo Romo Sánchez	`usuario_a`	Ingeniería mecánica	`mecanica`, `proyecto`
B	Alonso Malvar Ariza	`alonso`	Aviónica y software	`avionica`, `software`, `proyecto`
C	Ariadna Sánchez Villa	`ariadna`	Compras y configuración	`compras`, `proyecto`

El enunciado permite usar nombres reales o nombres acordados. Alonso y Ariadna usan su nombre como cuenta Linux. Para el miembro A usamos `usuario_a`, porque `guillermo` ya es la cuenta de administración del servidor creada en la P1, que pertenece al grupo `sudo`, y las cuentas de trabajo no deben tenerlo.

1.4. Entorno y topología

Tabla 2. Entorno de trabajo del grupo

Elemento	Valor
Servidor del grupo	PC de Guillermo (Windows) con Oracle VirtualBox
Máquina virtual	`server-asio-g3` · Ubuntu Server 26.04.1 LTS · núcleo 7.0.0-38 · x86_64
Red de la VM	NAT con reenvío de puertos (TCP 2222 del anfitrión al 22 de la VM)
Clientes	Portátil de Alonso (Windows) y de Ariadna (macOS) con el cliente OpenSSH
Cuenta de administración	`guillermo` (grupo `sudo`)
Cuentas de trabajo	`usuario_a`, `alonso` y `ariadna`, sin sudo
VMs individuales	Alonso (Windows, cuenta de administración `al`) y Ariadna (MacBook Air M2, arm64, cuenta de administración `adminasv`)
Paquetes añadidos	`acl` (`setfacl`, `getfacl`) y `tree`

```
 Portátil de Alonso  --+
                       +--> 10.168.8.56:2222  (PC de Guillermo, Windows + VirtualBox)
 Portátil de Ariadna --+          |
                                  |  NAT, reenvío de puertos 2222 -> 22
                                  v
                  VM server-asio-g3, 10.0.2.15:22 (sshd)
```

La VM sigue en modo NAT, como justificamos en la P1, porque asegura el acceso a Internet sin depender de la red del aula. El inconveniente es que desde fuera del anfitrión solo se llega a la VM a través de las reglas de reenvío, y en la P1 la regla de SSH escuchaba solo en `127.0.0.1`, es decir, únicamente desde el propio PC de Guillermo. Para que los otros dos portátiles pudieran conectarse cambiamos la regla para que escuche en todas las interfaces del anfitrión y abrimos ese puerto en el firewall de Windows, limitado a la subred local:

```
# PowerShell como administrador en el PC de Guillermo
VBoxManage controlvm "server-asio-g3" natpf1 delete ssh
VBoxManage controlvm "server-asio-g3" natpf1 "ssh,tcp,,2222,10.0.2.15,22"
New-NetFirewallRule -DisplayName "ASSIO SSH VM 2222" -Direction Inbound -Protocol TCP `
    -LocalPort 2222 -RemoteAddress LocalSubnet -Action Allow -Profile Any
ipconfig            # IPv4 del PC servidor en la Wi-Fi: 10.168.8.56
```

Al dejar vacío el campo de IP del anfitrión, VirtualBox escucha en todas sus interfaces. `-RemoteAddress LocalSubnet` hace que solo los equipos de la misma subred puedan usar el puerto, y `-Profile Any` es necesario porque las redes de la universidad suelen clasificarse como públicas. Al terminar la sesión la regla se elimina con `Remove-NetFirewallRule -DisplayName "ASSIO SSH VM 2222"`. Dentro de la VM comprobamos que SSH estaba activo y aceptaba contraseñas:

```
ip -4 addr show enp0s3
systemctl is-active ssh.socket ssh
sudo ss -tlnp | grep ':22'
sudo sshd -T | grep -i passwordauthentication
```

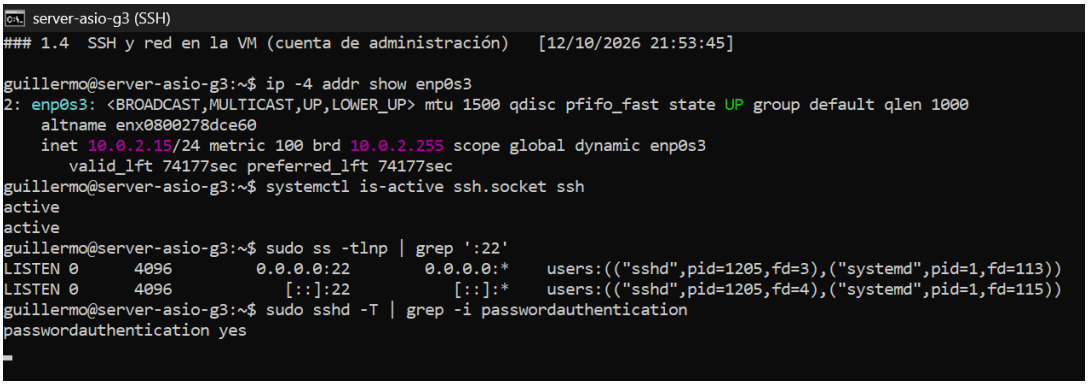


Ilustración 1. Comprobación de la red y del servicio SSH dentro de la VM con la cuenta guillermo

La VM tiene la IP `10.0.2.15`, el servicio `ssh` está activo, `sshd` escucha en el puerto 22 (IPv4 e IPv6) y la autenticación por contraseña está habilitada. Con todo preparado, el 20/10/2026 los tres equipos estaban en la misma red Wi-Fi y Alonso y Ariadna se conectaron al servidor con su propia cuenta:

```
# Ariadna (macOS)
nc -vz 10.168.8.56 2222
ssh -p 2222 ariadna@10.168.8.56
whoami ; hostname ; id

# Alonso (Windows, PowerShell)
Test-NetConnection 10.168.8.56 -Port 2222
ssh -p 2222 alonso@10.168.8.56
whoami ; hostname ; id
```



Ilustración 2. Conexión de Ariadna desde su portátil al servidor del grupo con la cuenta ariadna

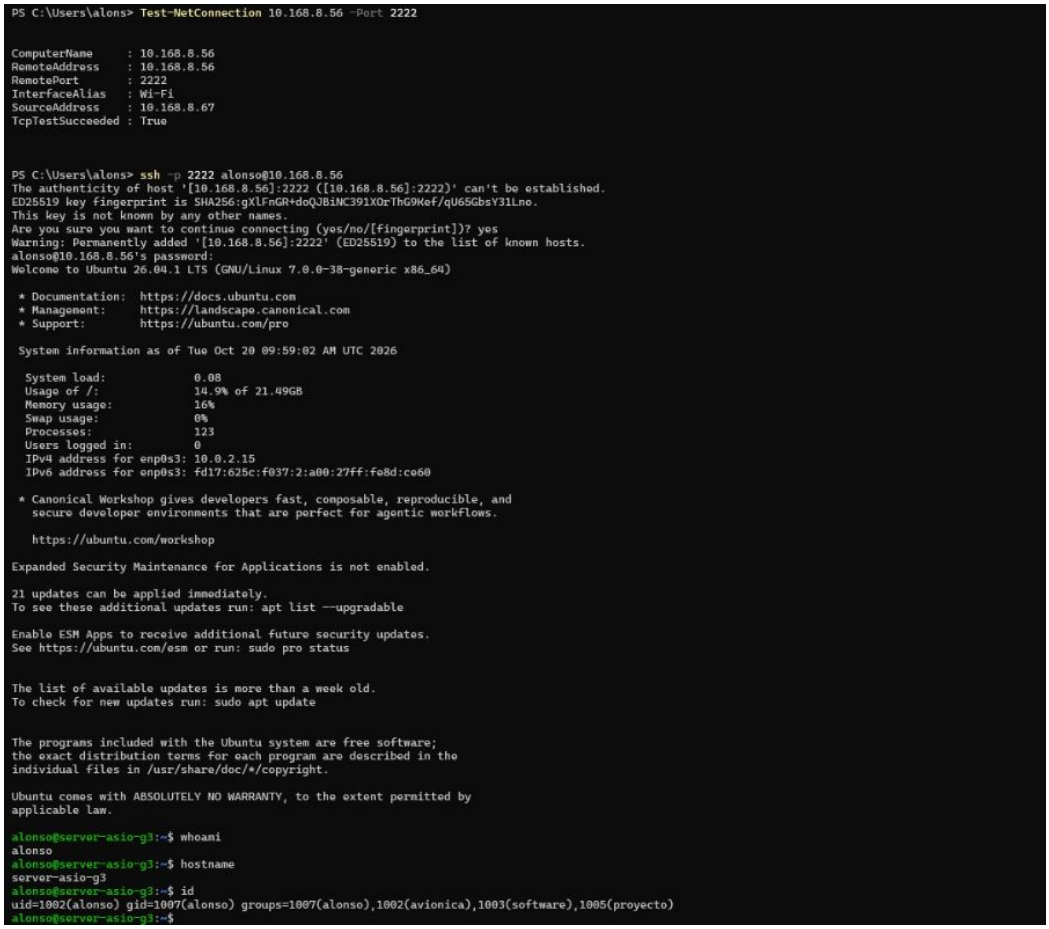


Ilustración 3. Conexión de Alonso desde su portátil al servidor del grupo con la cuenta alonso

En ambos casos la prueba de puerto da resultado positivo (`succeeded` y `TcpTestSucceeded : True`), la sesión se abre en `server-asio-g3` y `id` muestra que cada cuenta ya pertenece a los grupos de su rol: `ariadna` a `compras` y `proyecto`, y `alonso` a `avionica`, `software` y `proyecto`. La primera conexión de Alonso pide aceptar la huella del servidor, lo que es normal la primera vez que un cliente se conecta a un equipo.

2. Primer acceso al sistema e identificación básica

Antes de modificar nada, cada uno comprobamos quién somos dentro del sistema, en qué máquina estamos y cómo se organiza la jerarquía de directorios. En el servidor del grupo Guillermo lo hizo con la cuenta de administración y repetimos la secuencia con las cuentas de Alonso y Ariadna; además, Alonso y Ariadna lo hicieron en su propia VM.

```
whoami
hostname
hostnamectl
pwd
ls -l /
ls -la /home
ls -la /root          # sin sudo: Permission denied
sudo ls -la /root     # con sudo: muestra el contenido
```

Tabla 3. Comandos de identificación y lo que muestran

Comando	Qué muestra	Resultado esperado
`whoami`	Usuario efectivo de la sesión	El nombre de la cuenta con la que hemos entrado
`hostname` / `hostnamectl`	Nombre del equipo; `hostnamectl` añade sistema, núcleo, arquitectura y virtualización	Ubuntu 26.04.1 LTS, virtualización oracle
`pwd`	Directorio de trabajo actual	`/home/<cuenta>` al iniciar sesión
`ls -l /`	Directorios de primer nivel del sistema	`bin`, `boot`, `etc`, `home`, `root`, `srv`, `usr`, `var`...
`ls -la /home`	Carpetas personales de los usuarios normales	Una entrada por usuario
`ls -la /root`	Carpeta personal del superusuario	Sin sudo: Permission denied; con sudo: su contenido

2.1. Servidor del grupo

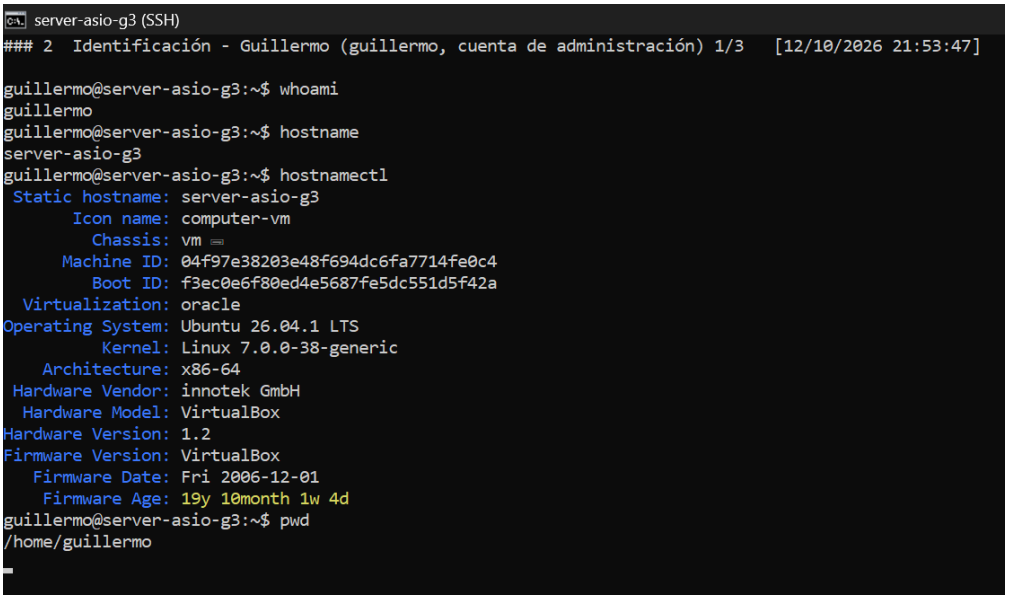


Ilustración 4. Identificación en el servidor con la cuenta de administración guillermo (1/3)

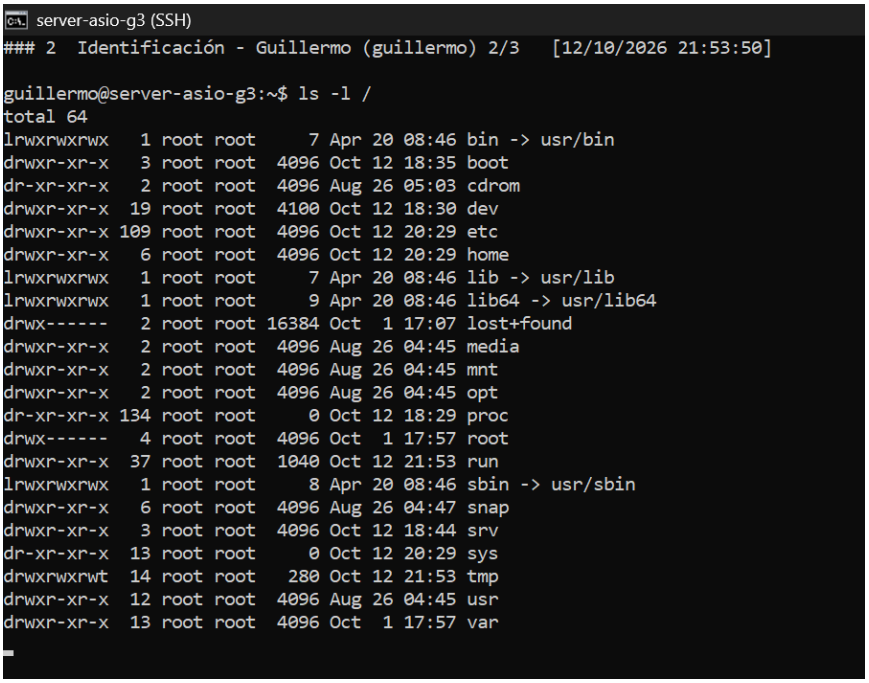


Ilustración 5. Jerarquía de primer nivel del servidor con ls -l / (2/3)

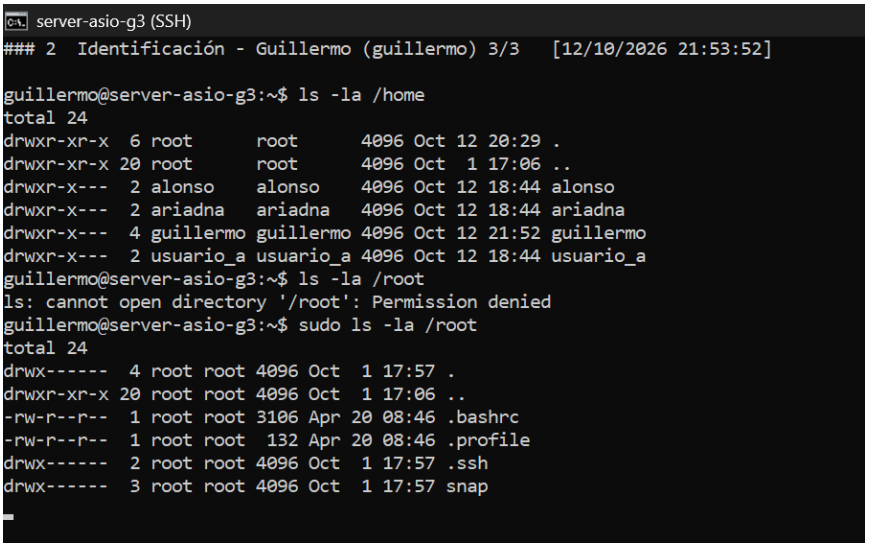


Ilustración 6. Contenido de /home y acceso a /root sin y con sudo en el servidor (3/3)

`whoami` devuelve `guillermo`, `hostname` y `hostnamectl` confirman que estamos en `server-asio-g3` con Ubuntu 26.04.1 LTS virtualizado en VirtualBox, y `pwd` muestra que empezamos en `/home/guillermo`. En `/home` aparece una carpeta por cuenta, y `ls -la /root` sin sudo da «Permission denied» mientras que con sudo sí muestra el contenido.

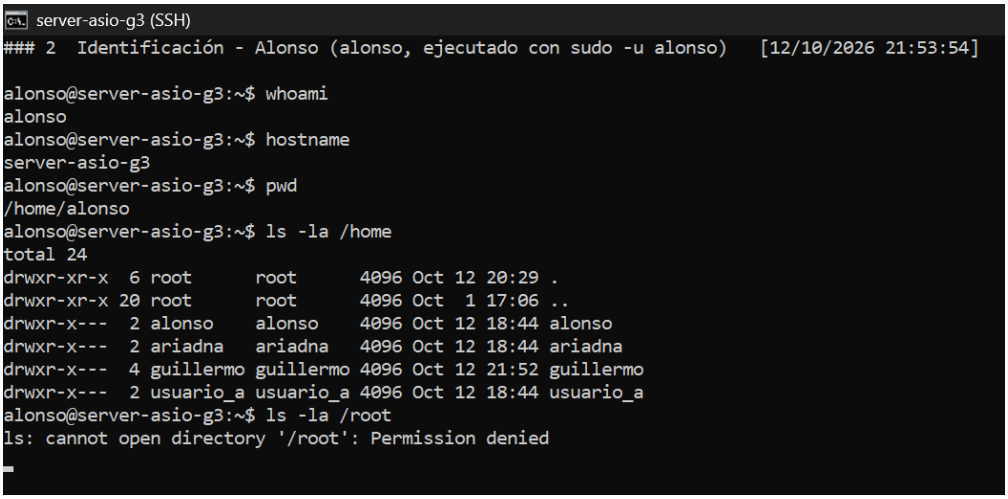


Ilustración 7. Identificación en el servidor con la cuenta alonso

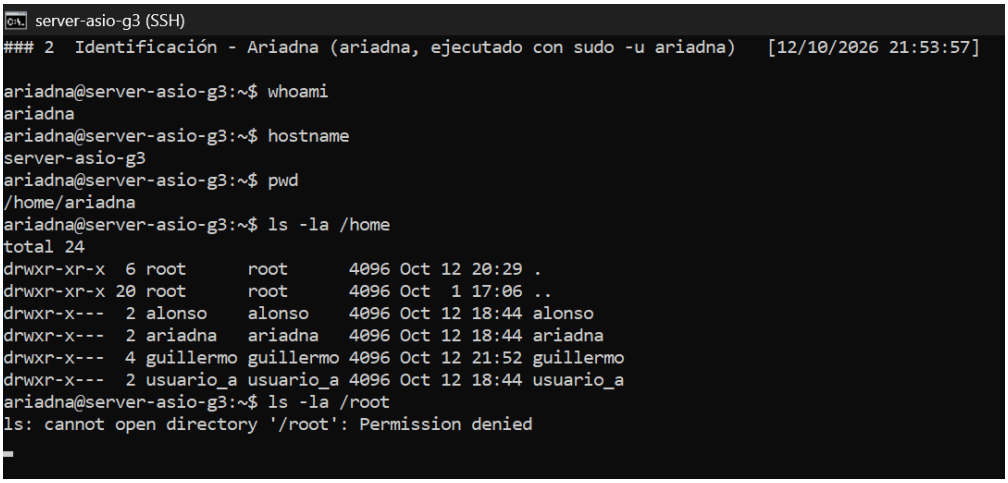


Ilustración 8. Identificación en el servidor con la cuenta ariadna

Con las cuentas de trabajo el resultado cambia en lo esperado: `whoami` devuelve `alonso` o `ariadna`, `pwd` su propia carpeta y `/root` queda cerrado, porque estas cuentas no tienen sudo.

2.2. Máquina virtual de Alonso

```
al@server-assio-03:~$ whoami
al
al@server-assio-03:~$ hostname
server-assio-03
al@server-assio-03:~$ hostnamectl
 Static hostname: server-assio-03
       Icon name: computer-vm
         Chassis: vm
      Machine ID: 7470735bf198465dbde86acd9619a10a
         Boot ID: 69a8a892b00249d081a0a29ec25cf752
  Virtualization: oracle
Operating System: Ubuntu 26.04.1 LTS
          Kernel: Linux 7.0.0-38-generic
    Architecture: x86-64
 Hardware Vendor: innotek GmbH
  Hardware Model: VirtualBox
Hardware Version: 1.2
Firmware Version: VirtualBox
   Firmware Date: Fri 2006-12-01
    Firmware Age: 19y 10month 2w 3d
al@server-assio-03:~$ pwd
/home/al
al@server-assio-03:~$ ls -l /
total 64
lrwxrwxrwx   1 root root     7 Apr 20 08:46 bin -> usr/bin
drwxr-xr-x   3 root root  4096 Oct  6 11:46 boot
dr-xr-xr-x   2 root root  4096 Aug 26 05:03 cdrom
drwxr-xr-x  19 root root  4120 Oct 18 15:37 dev
drwxr-xr-x 109 root root  4096 Oct  1 14:34 etc
drwxr-xr-x   4 root root  4096 Oct  1 14:12 home
lrwxrwxrwx   1 root root     7 Apr 20 08:46 lib -> usr/lib
lrwxrwxrwx   1 root root     9 Apr 20 08:46 lib64 -> usr/lib64
drwx------   2 root root 16384 Oct  1 14:04 lost+found
drwxr-xr-x   2 root root  4096 Aug 26 04:45 media
drwxr-xr-x   2 root root  4096 Aug 26 04:45 mnt
drwxr-xr-x   2 root root  4096 Aug 26 04:45 opt
dr-xr-xr-x 127 root root     0 Oct 18 15:37 proc
drwx------   4 root root  4096 Oct  1 14:34 root
drwxr-xr-x  33 root root   920 Oct 18 15:43 run
lrwxrwxrwx   1 root root     8 Apr 20 08:46 sbin -> usr/sbin
drwxr-xr-x   6 root root  4096 Aug 26 04:47 snap
drwxr-xr-x   2 root root  4096 Aug 26 04:45 srv
dr-xr-xr-x  13 root root     0 Oct 18 15:43 sys
drwxrwxrwt  12 root root   240 Oct 18 15:48 tmp
drwxr-xr-x  12 root root  4096 Aug 26 04:45 usr
drwxr-xr-x  13 root root  4096 Oct  1 14:34 var
```

Ilustración 9. Identificación en la VM de Alonso con su cuenta de administración al

```
al@server-assio-03:~$ ls -la /home
total 28
drwxr-xr-x  4 root root  4096 Oct  1 14:12 .
drwxr-xr-x 20 root root  4096 Oct  1 14:04 ..
drwxr-x---  4 al   al    4096 Oct 18 15:43 al
drwx------  2 root root 16384 Oct  1 14:04 lost+found
al@server-assio-03:~$ ls -la /root
ls: cannot open directory '/root': Permission denied
al@server-assio-03:~$ sudo ls -la /root
total 24
drwx------  4 root root 4096 Oct  1 14:34 .
drwxr-xr-x 20 root root 4096 Oct  1 14:04 ..
-rw-r--r--  1 root root 3106 Apr 20 08:46 .bashrc
-rw-r--r--  1 root root  132 Apr 20 08:46 .profile
drwx------  2 root root 4096 Oct  1 14:34 .ssh
drwx------  3 root root 4096 Oct  1 14:34 snap
al@server-assio-03:~$
```

Ilustración 10. Contenido de /home y acceso a /root en la VM de Alonso

2.3. Máquina virtual de Ariadna

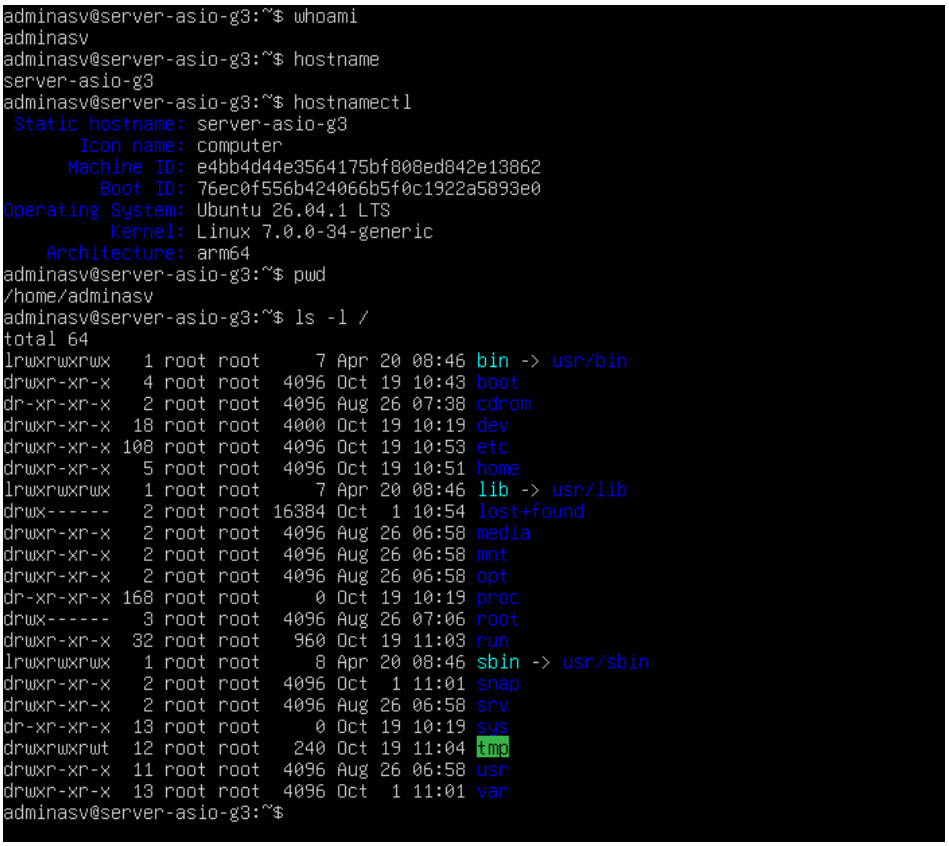


Ilustración 11. Identificación en la VM de Ariadna con su cuenta de administración adminasv

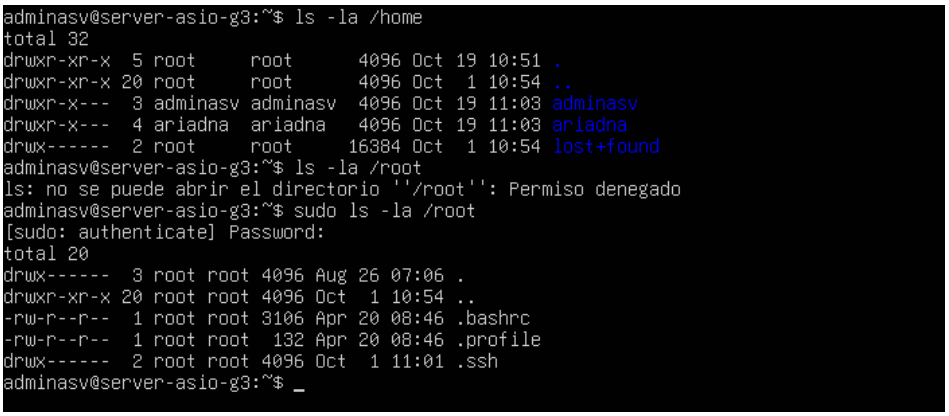


Ilustración 12. Contenido de /home y acceso a /root en la VM de Ariadna

En las tres máquinas se ve la misma jerarquía: `/` es la raíz de todo el sistema de archivos, `/home` contiene las carpetas personales de los usuarios normales y `/root` es la carpeta personal del superusuario, fuera de `/home` y con permisos 700, por eso un usuario normal no puede listarla. `/srv` está reservado para los datos que sirve el sistema, y por eso es donde colocamos la estructura de la empresa. La única diferencia entre las VMs es la arquitectura: la de Ariadna, en un Mac con chip M2, es arm64.

2.4. Por qué no se usa root como cuenta habitual

La cuenta root no tiene ninguna restricción: el núcleo no le aplica comprobaciones de permisos, así que cualquier error al escribir (un `rm -r` sobre la ruta equivocada o un `chmod -R` mal dirigido) se ejecuta sin confirmación y puede dejar el sistema inutilizable. Además, cualquier programa lanzado como root hereda todos sus privilegios, de modo que un fallo de seguridad en ese programa compromete todo el sistema, y si todos trabajásemos como root no quedaría registro de quién hizo cada cambio.

Trabajar con una cuenta normal y elevar privilegios con `sudo` solo cuando hace falta aplica el principio de mínimo privilegio: `sudo` solo funciona para las cuentas autorizadas (en Ubuntu, el grupo `sudo`), pide confirmación antes de cada operación de administración y registra cada orden en `/var/log/auth.log`. Por estas razones Ubuntu deja la cuenta root bloqueada por defecto.

3. Creación de la estructura de la empresa

La estructura se crea con un solo `mkdir`. La opción `-p` crea los directorios intermedios que falten y no da error si alguno ya existe, lo que hace el comando repetible. Las llaves `{a,b,c}` son la expansión de llaves de Bash: el intérprete genera una ruta por cada elemento antes de ejecutar `mkdir`. Como `/srv` pertenece a root, hace falta `sudo`.

```
sudo apt update && sudo apt install -y tree acl
sudo mkdir -p /srv/aurora6/ingenieria/{mecanica,avionica,software} \
              /srv/aurora6/compras/{proveedores,pedidos} \
              /srv/aurora6/contabilidad/facturas \
              /srv/aurora6/documentacion/comun \
              /srv/aurora6/confidencial \
              /srv/aurora6/intercambio/{incoming,approved,quarantine}
tree -d /srv/aurora6
```

`tree -d` muestra solo los directorios en forma de árbol y su última línea da el total: «17 directories», porque cuenta el propio `/srv/aurora6` más 6 directorios de primer nivel y 10 subdirectorios. En el mismo paso instalamos el paquete `acl`, que trae `setfacl` y `getfacl` para la sección 6.

En el servidor la estructura es un recurso compartido, así que la creó una sola vez Guillermo desde la cuenta de administración. Alonso y Ariadna la comprobaron después con `tree -d` desde su propia cuenta: cada uno ve los directorios a los que tiene acceso y el resto aparece como «error opening dir», lo que ya anticipa la política de la sección 6.

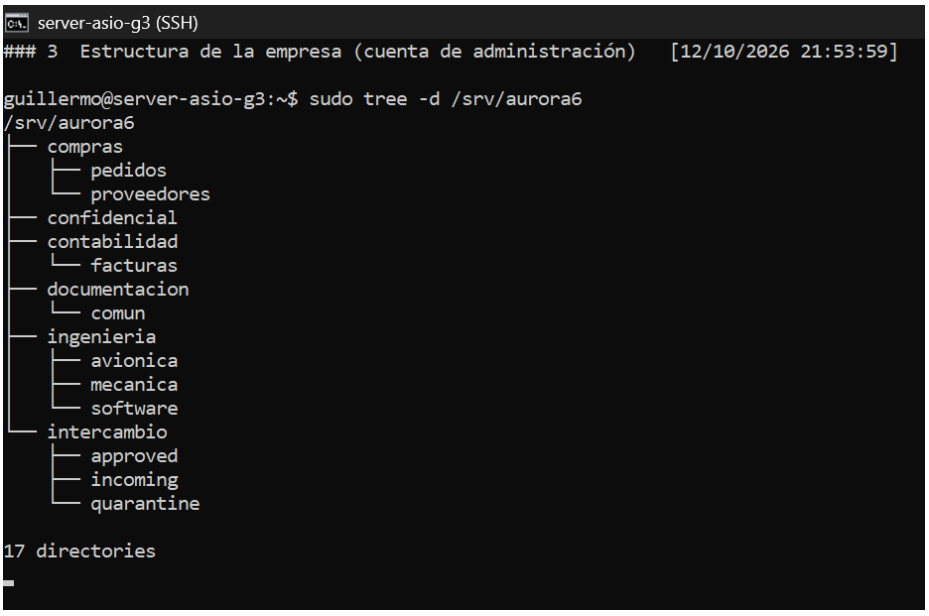


Ilustración 13. Estructura de /srv/aurora6 en el servidor vista desde la cuenta de administración

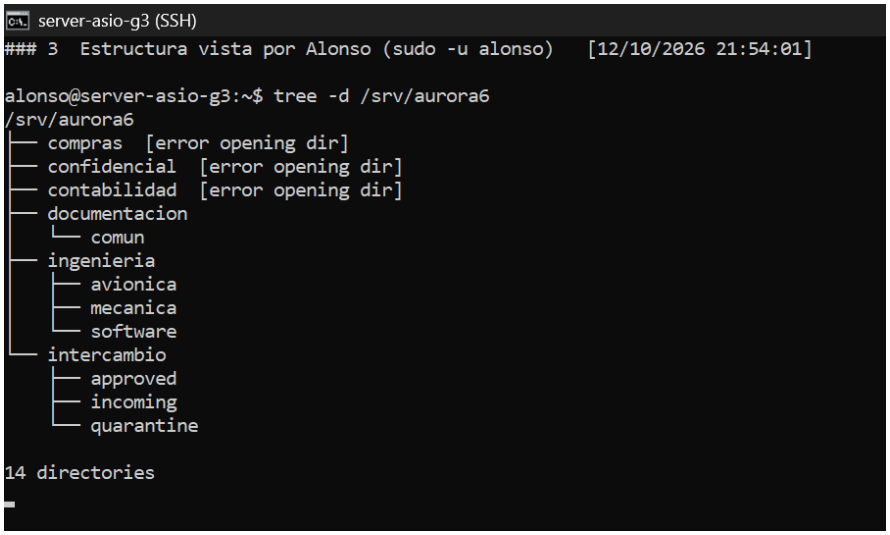


Ilustración 14. Estructura del servidor vista con la cuenta alonso

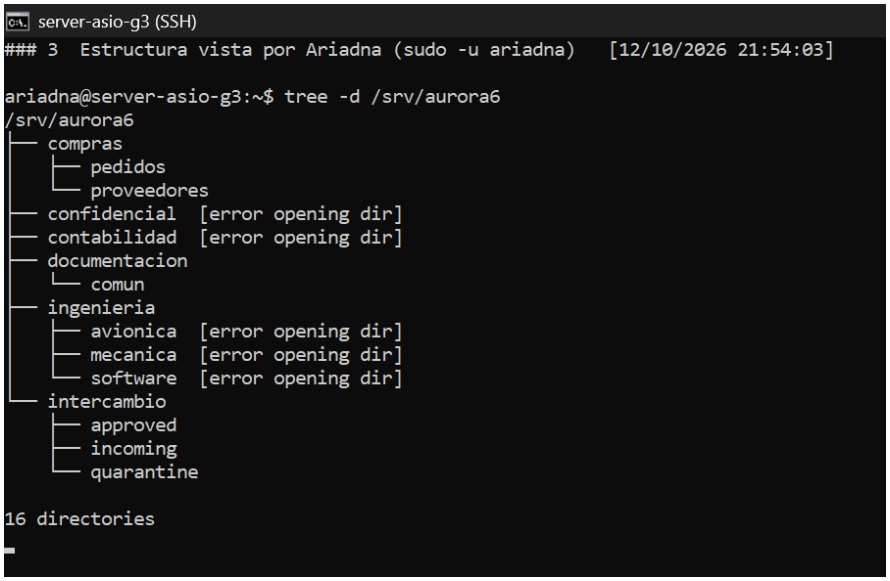


Ilustración 15. Estructura del servidor vista con la cuenta ariadna

En sus propias VMs, Alonso y Ariadna crearon la estructura completa desde su cuenta de administración con el mismo comando, y el resultado es idéntico:

```
al@server-assio-03:~$ sudo bash p2_servidor.sh estructura

==> Creando la estructura de /srv/aurora6
/srv/aurora6
├── compras
│   ├── pedidos
│   └── proveedores
├── confidencial
├── contabilidad
│   └── facturas
├── documentacion
│   └── comun
├── ingenieria
│   ├── avionica
│   ├── mecanica
│   └── software
└── intercambio
    ├── approved
    ├── incoming
    └── quarantine

17 directories
al@server-assio-03:~$
```

Ilustración 16. Estructura de /srv/aurora6 creada en la VM de Alonso

```
adminasv@server-asio-g3:~$ sudo bash p2.sh estructura

==> Creando la estructura de /srv/aurora6
+ mkdir -p /srv/aurora6/ingenieria/mecanica /srv/aurora6/ingenieria/avionica /srv/aurora6/ingenieria/software /srv/aurora6/compras/proveedores /srv/aurora6/comp
ras/pedidos /srv/aurora6/contabilidad/facturas /srv/aurora6/documentacion/comun /srv/aurora6/confidencial /srv/aurora6/intercambio/incoming /srv/aurora6/interca
mbio/approved /srv/aurora6/intercambio/quarantine
+ tree -d /srv/aurora6
/srv/aurora6
├── compras
│   ├── pedidos
│   └── proveedores
├── confidencial
├── contabilidad
│   └── facturas
├── documentacion
│   └── comun
├── ingenieria
│   ├── avionica
│   ├── mecanica
│   └── software
└── intercambio
    ├── approved
    ├── incoming
    └── quarantine

17 directories
adminasv@server-asio-g3:~$
```

Ilustración 17. Estructura de /srv/aurora6 creada en la VM de Ariadna

4. Usuarios, grupos y roles

4.1. Creación de grupos y cuentas

```
sudo groupadd mecanica
sudo groupadd avionica
sudo groupadd software
sudo groupadd compras
sudo groupadd proyecto
sudo useradd -m -s /bin/bash usuario_a
sudo useradd -m -s /bin/bash alonso
sudo useradd -m -s /bin/bash ariadna
sudo passwd usuario_a
sudo passwd alonso
sudo passwd ariadna
```

`groupadd` crea cada grupo en `/etc/group` con un GID libre. `useradd -m` crea la carpeta personal copiando `/etc/skel` y `-s /bin/bash` fija el intérprete (sin esa opción sería `/bin/sh`). En Ubuntu, `useradd` crea además un grupo privado con el mismo nombre que el usuario, que queda como grupo primario. Preferimos `useradd` a `adduser` porque no es interactivo y por tanto se puede repetir igual dentro de un script. La contraseña se asigna con `passwd`, que la guarda cifrada en `/etc/shadow`; cada integrante escribió la suya.

4.2. Pertenencia a los grupos de departamento

```
sudo usermod -aG mecanica,proyecto usuario_a
sudo usermod -aG avionica,software,proyecto alonso
sudo usermod -aG compras,proyecto ariadna
```

`usermod -G` fija la lista de grupos complementarios y `-a` hace que se añadan a los que ya tenía la cuenta. Sin `-a` el usuario perdería cualquier grupo que no estuviera en la lista, un error clásico que puede llegar a quitar el grupo `sudo` a una cuenta de administración. Usamos los departamentos como grupos complementarios y mantenemos el grupo privado como primario, porque Alonso pertenece a dos departamentos y una cuenta solo puede tener un grupo primario.

Las cuentas de trabajo no se añaden al grupo `sudo`, y esta decisión es necesaria para la sección 6: si una cuenta de trabajo pudiera usar `sudo`, podría entrar en confidencial o en cualquier otro directorio y la política no tendría efecto.

4.3. Comprobación

```
id usuario_a ; groups usuario_a
id alonso ; groups alonso
id ariadna ; groups ariadna
getent group mecanica avionica software compras proyecto
```

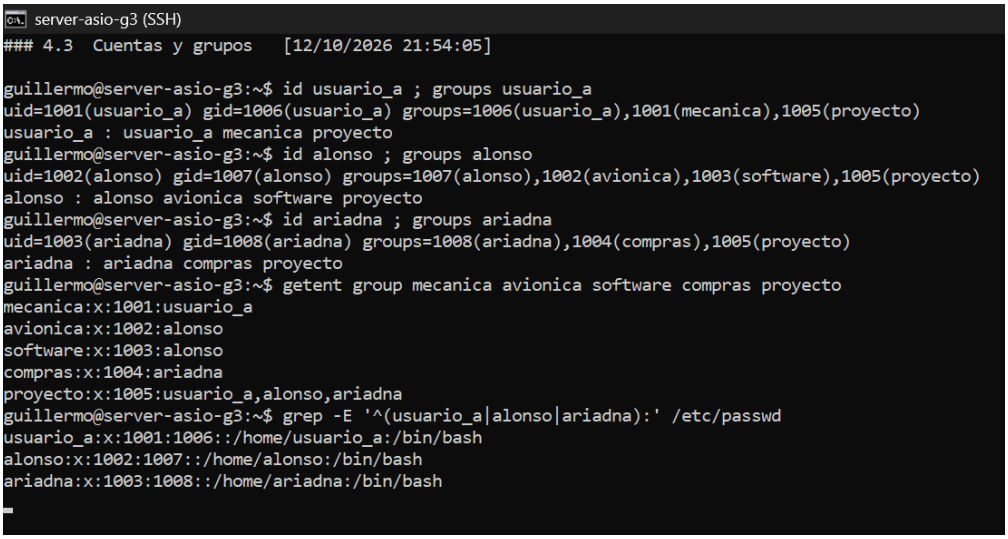


Ilustración 18. Cuentas, grupos y pertenencias comprobadas en el servidor

Tabla 4. Pertenencia de las cuentas de trabajo a los grupos

Cuenta	Grupo primario	Grupos complementarios	sudo
`usuario_a`	`usuario_a`	`mecanica`, `proyecto`	No
`alonso`	`alonso`	`avionica`, `software`, `proyecto`	No
`ariadna`	`ariadna`	`compras`, `proyecto`	No

`id` muestra el UID, el grupo primario y todos los grupos con su número; `groups` solo los nombres, y `getent group` muestra en el último campo los miembros de cada grupo. El resultado coincide con la tabla de roles. Hay que tener en cuenta que los grupos se cargan al iniciar sesión: si un usuario tenía una sesión abierta cuando se le añadió a un grupo, en esa sesión no lo verá hasta que salga y vuelva a entrar.

En las VMs individuales se hizo lo mismo. En la de Alonso, el bloque de usuarios del script muestra el `id` de cada cuenta y a continuación se asignan las contraseñas:

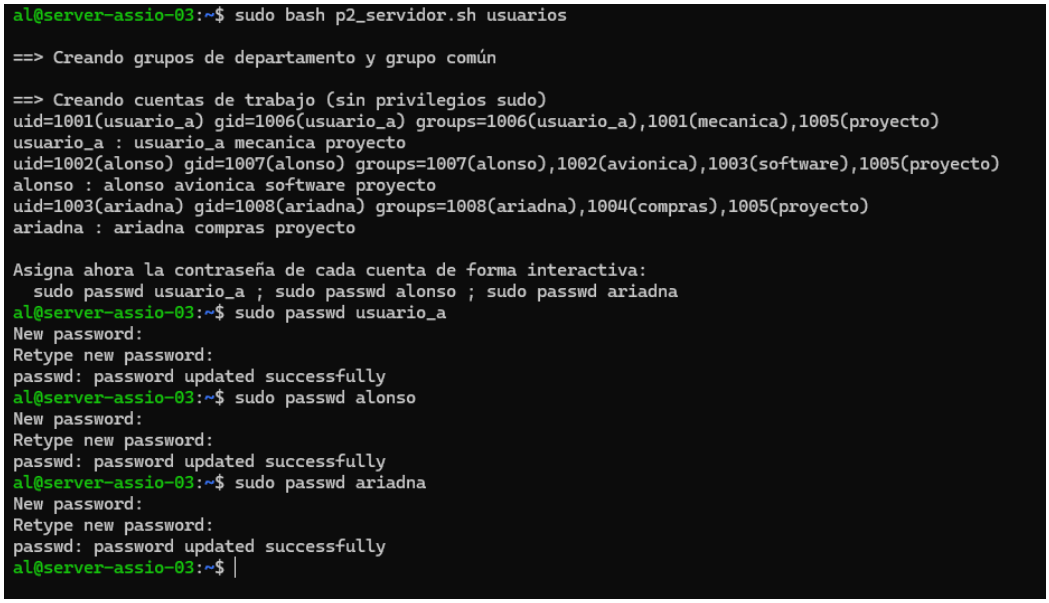


Ilustración 19. Creación de cuentas y grupos en la VM de Alonso

En la VM de Ariadna la cuenta `ariadna` ya existía desde la P1 y era la de administración, así que primero creó una cuenta de administración aparte, `adminasv`, y quitó a `ariadna` del grupo `sudo` para que quedase como cuenta de trabajo:

```
sudo adduser adminasv
sudo usermod -aG sudo adminasv
sudo gpasswd -d ariadna sudo
```

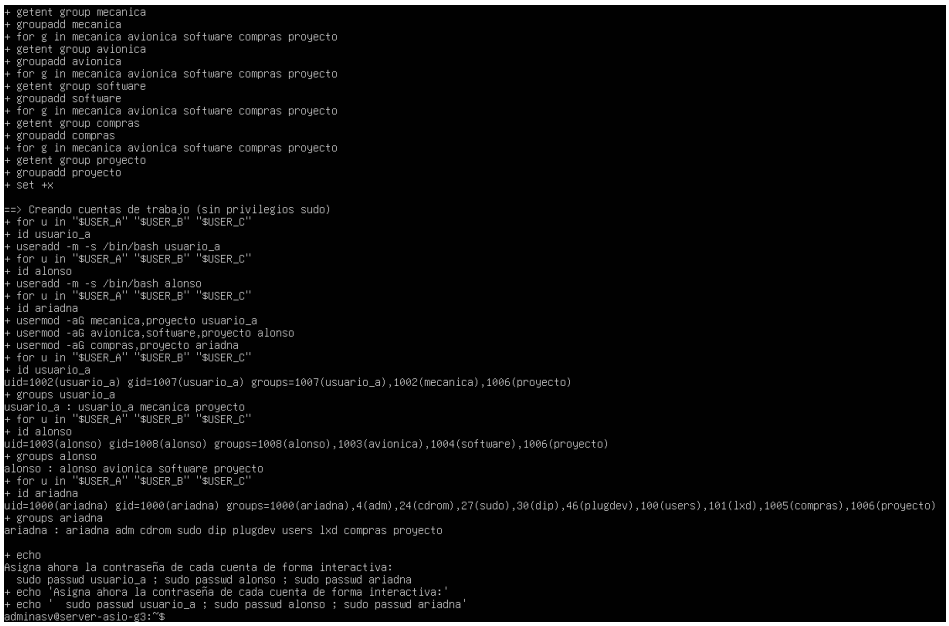


Ilustración 20. Creación de grupos y cuentas en la VM de Ariadna con cada comando visible



Ilustración 21. Grupos y pertenencias comprobados en la VM de Ariadna

5. Permisos tradicionales (DAC) y configuración 751

```
sudo chown root:proyecto /srv/aurora6/documentacion
sudo chmod 751 /srv/aurora6/documentacion
ls -ld /srv/aurora6/documentacion
```

`chown root:proyecto` deja a root como propietario y asigna `proyecto` como grupo. `chmod 751` fija los permisos en notación octal y `ls -ld` muestra los permisos del propio directorio (la `-d` evita que liste su contenido). En las tres máquinas el resultado es `drwxr-x--x` con propietario `root` y grupo `proyecto`, como pide el enunciado.

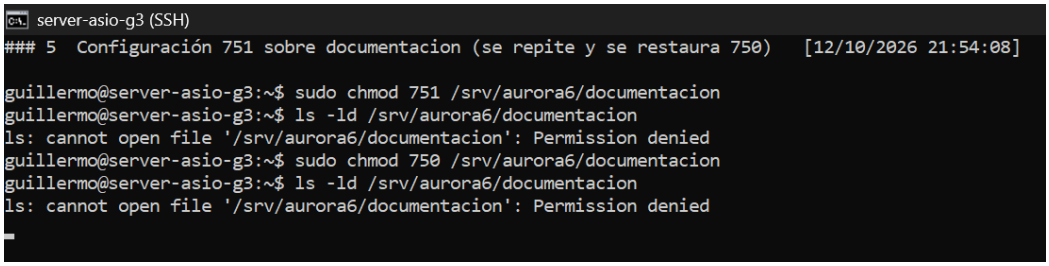


Ilustración 22. Configuración 751 sobre documentacion en el servidor

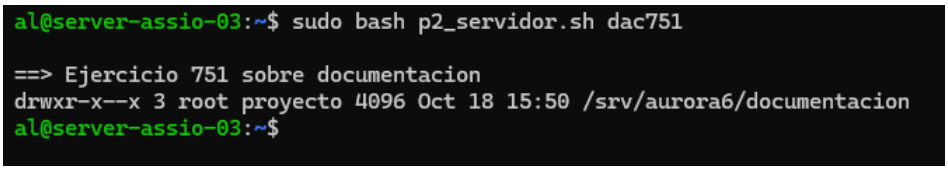


Ilustración 23. Configuración 751 sobre documentacion en la VM de Alonso

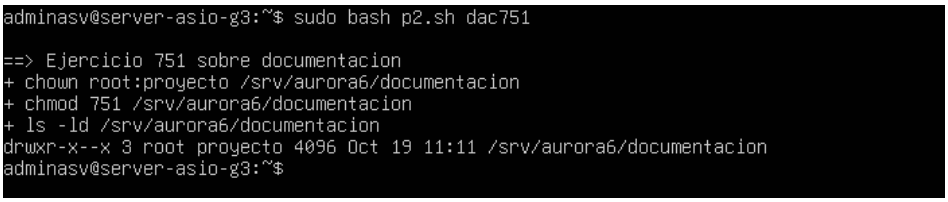


Ilustración 24. Configuración 751 sobre documentacion en la VM de Ariadna

5.1. Interpretación de 7, 5 y 1

Cada cifra octal es la suma de tres bits: lectura r = 4, escritura w = 2 y ejecución x = 1. La primera cifra es para el propietario, la segunda para el grupo y la tercera para el resto de usuarios. La `d` inicial indica que es un directorio.

Tabla 5. Descomposición del permiso 751 sobre documentacion

Cifra	Clase	Bits	Qué permite en este directorio
7 = 4+2+1	Propietario (`root`)	`rwx`	Listar el contenido, crear, borrar y renombrar entradas y entrar en él
5 = 4+0+1	Grupo (`proyecto`)	`r-x`	Listar y entrar, pero no crear ni borrar nada
1 = 0+0+1	Otros	`--x`	Atravesarlo para llegar a una ruta que ya conocen, sin poder listarlo

5.2. Qué implica el permiso x en un directorio

En un directorio los permisos no significan lo mismo que en un archivo. La `r` permite leer la lista de nombres (`ls`); la `w` permite modificar esa lista, es decir, crear, borrar o renombrar entradas, y solo funciona si además se tiene `x`; y la `x` es el permiso de paso: permite entrar con `cd` y acceder a lo que hay dentro por su nombre. Por eso `x` es imprescindible en todos los directorios de una ruta para llegar a un archivo.

Con solo `x`, como los «otros» en 751, un usuario puede abrir `documentacion/comun/archivo.txt` si conoce el nombre y el archivo se lo permite, pero `ls documentacion` le devuelve Permission denied. Con solo `r`, en cambio, vería los nombres pero no podría entrar ni abrir nada. Esta misma idea es la que usamos después en contabilidad con una ACL de solo paso.

El 751 era una configuración para practicar la interpretación. En la política definitiva `documentacion` pasa a 750, porque no hay motivo para que usuarios ajenos al proyecto puedan atravesarlo.

6. Política de acceso por roles y ACLs

6.1. Traducción de la política

El primer paso fue convertir la política del enunciado en una matriz de accesos por directorio y por usuario. Para llegar a un directorio hace falta además `x` en todos los de la ruta, así que también tuvimos en cuenta los niveles intermedios.

Tabla 6. Matriz de accesos derivada del enunciado

Directorio	usuario_a (A)	alonso (B)	ariadna (C)
`ingenieria/mecanica`	`rwx`	`r-x`	Sin acceso
`ingenieria/avionica`	`r-x`	`rwx`	Sin acceso
`ingenieria/software`	Sin acceso	`rwx`	Sin acceso
`compras` (proveedores, pedidos)	Sin acceso	Sin acceso	`rwx`
`contabilidad`	Sin acceso	Sin acceso	Solo paso (`--x`)
`contabilidad/facturas`	Sin acceso	Sin acceso	`r-x`
`documentacion/comun`	`r-x`	`r-x`	`r-x`
`confidencial`	Sin acceso	Sin acceso	Sin acceso

Donde el enunciado no dice nada (por ejemplo, el acceso de A a software o de A y B a proveedores) hemos aplicado el principio de mínimo privilegio y no hemos dado acceso. En el caso de C sobre ingeniería, el enunciado solo exige que no pueda modificar; negarle también la lectura cumple ese requisito y es más restrictivo. De la matriz sale la configuración final:

Tabla 7. Propietarios, permisos y ACL finales

Ruta	Propietario:grupo	Modo	ACL	Configura
`/srv/aurora6`, `ingenieria`, `documentacion`	`root:proyecto`	`750`	No	Administración
`ingenieria/mecanica`	`root:mecanica`	`2770`	`u:alonso:r-x` (+ por defecto)	A
`ingenieria/avionica`	`root:avionica`	`2770`	`u:usuario_a:r-x` (+ por defecto)	B
`ingenieria/software`	`root:software`	`2770`	No	B
`compras`, `proveedores`, `pedidos`	`root:compras`	`2770`	No	C
`documentacion/comun`	`root:proyecto`	`2750`	No	C
`contabilidad`	`root:root`	`750`	`u:ariadna:--x`	C
`contabilidad/facturas`	`root:root`	`750`	`u:ariadna:r-x` (+ por defecto)	C
`confidencial`	`root:root`	`700`	No	Administración
`intercambio` y subdirectorios	`root:proyecto`	`750`	No	Administración

6.2. Criterios de diseño

Los grupos de departamento son el mecanismo principal. Cada directorio de departamento pertenece a root y al grupo del departamento con permisos 770, de modo que el grupo tiene

control total y el resto ninguno. Dejamos a root como propietario en vez de a la cuenta del integrante para que el directorio no dependa de una persona: si mañana entra otro ingeniero mecánico, basta con añadirlo al grupo `mecanica`.

El 2 inicial es el bit setgid. En un directorio hace que los archivos y subdirectorios creados dentro hereden el grupo del directorio en vez del grupo primario de quien los crea. Sin él, un archivo que `usuario_a` creara en mecánica pertenecería a su grupo privado y el resto del departamento no podría trabajar con él. En `ls -ld` aparece como una `s` en la posición de ejecución del grupo (`drwxrws---`).

Los niveles intermedios (`/srv/aurora6`, `ingenieria`, `documentacion`, `intercambio`) quedan a 750 para el grupo `proyecto`, para que los tres integrantes puedan llegar a sus directorios y nadie más del sistema. `confidencial` queda a 700 de root: ninguna cuenta de trabajo es propietaria ni está en el grupo root y los otros no tienen permisos, así que solo la cuenta de administración mediante `sudo` puede gestionarlo.

6.3. Cuándo es necesaria una ACL y qué aporta

El modelo propietario/grupo/otros solo permite dar permisos a un usuario y a un grupo por objeto. En `mecanica` el grupo ya está ocupado por `mecanica` (`rwx`) y además necesitamos que Alonso tenga `r-x` sin que los otros tengan nada, porque Ariadna no debe entrar. Con DAC solo habría dos salidas: dar `r-x` a los otros, lo que abriría el directorio también a Ariadna, o crear un grupo artificial con A y B, lo que daría a B los mismos derechos que a A. Ninguna cumple la política. Lo mismo ocurre en `avionica` con A y en `contabilidad/facturas` con C, ya que contabilidad debe seguir cerrada para A y B.

La ACL extendida añade entradas para usuarios o grupos concretos sin cambiar el resto: `setfacl -m u:alonso:r-x mecanica` da lectura y paso solo a Alonso. La ventaja frente a propietario/grupo/otros es la granularidad: permisos distintos para varios usuarios sobre el mismo recurso, sin grupos ficticios y sin abrirlo a todos. Cuando un directorio tiene ACL, `ls -l` muestra un `+` al final de los permisos y los bits del grupo pasan a mostrar la máscara (`mask`), que es el máximo que pueden tener los usuarios y grupos nombrados. Si después un `chmod` reduce los permisos del grupo, la máscara baja y la ACL deja de tener efecto aunque siga apareciendo; por eso aplicamos siempre `chmod` antes de `setfacl` y comprobamos con `getfacl`, que marca con `#effective:` cuando la máscara recorta un permiso.

En los directorios con ACL definimos también una ACL por defecto (`setfacl -d`). La ACL de acceso solo afecta al directorio, mientras que la ACL por defecto se copia a los archivos y subdirectorios nuevos. Sin ella, un archivo que A creara en mecánica no tendría la entrada de Alonso y este no podría leerlo, aunque sí listar el directorio. Un caso especial es `contabilidad`: Ariadna necesita llegar a `facturas`, pero el enunciado no le da acceso al resto de la contabilidad, así que con `u:ariadna:--x` puede atravesar el directorio sin poder listarlo, que es el permiso de paso de la sección 5.

6.4. Configuración realizada por cada integrante

Siguiendo el reparto del enunciado, cada integrante preparó su parte de la política. Todo se aplica desde la cuenta de administración, porque las cuentas de trabajo no tienen sudo. Antes, la cuenta de administración fija los permisos de los niveles comunes:

```
sudo chown root:proyecto /srv/aurora6 /srv/aurora6/ingenieria /srv/aurora6/documentacion
/srv/aurora6/intercambio
sudo chmod 750 /srv/aurora6 /srv/aurora6/ingenieria /srv/aurora6/documentacion
/srv/aurora6/intercambio
sudo chown root:proyecto /srv/aurora6/intercambio/{incoming,approved,quarantine}
sudo chmod 750 /srv/aurora6/intercambio/{incoming,approved,quarantine}
sudo chown root:root /srv/aurora6/confidencial
sudo chmod 700 /srv/aurora6/confidencial
```

Miembro A, Guillermo: `mecanica` y ACL de lectura para B.

```
sudo chown root:mecanica /srv/aurora6/ingenieria/mecanica
sudo chmod 2770 /srv/aurora6/ingenieria/mecanica
sudo setfacl -m u:alonso:r-x /srv/aurora6/ingenieria/mecanica
sudo setfacl -d -m u::rwx,g::rwx,o::---,u:alonso:r-x /srv/aurora6/ingenieria/mecanica
getfacl /srv/aurora6/ingenieria/mecanica
```

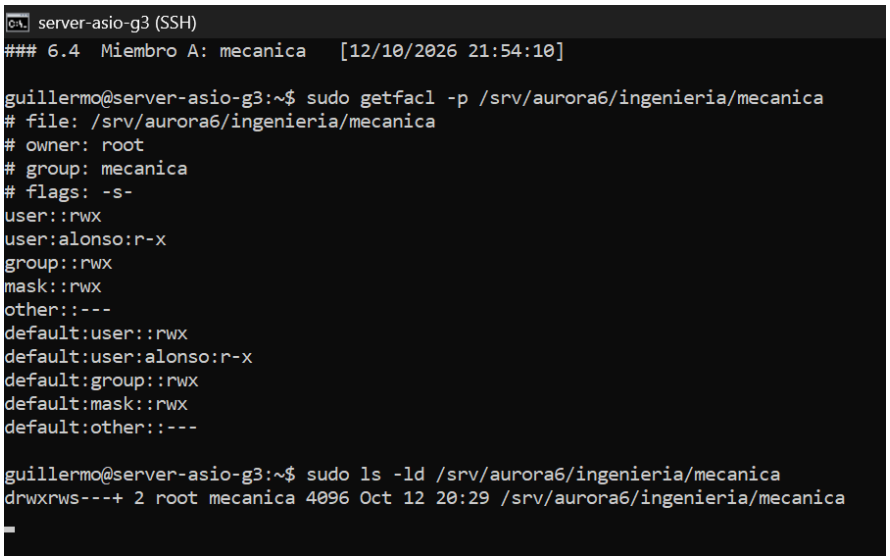


Ilustración 25. ACL de mecanica en el servidor (miembro A)

Miembro B, Alonso: `avionica`, `software` y ACL de lectura para A.

```
sudo chown root:avionica /srv/aurora6/ingenieria/avionica
sudo chmod 2770 /srv/aurora6/ingenieria/avionica
sudo setfacl -m u:usuario_a:r-x /srv/aurora6/ingenieria/avionica
sudo setfacl -d -m u::rwx,g::rwx,o::---,u:usuario_a:r-x /srv/aurora6/ingenieria/avionica
sudo chown root:software /srv/aurora6/ingenieria/software
sudo chmod 2770 /srv/aurora6/ingenieria/software
getfacl /srv/aurora6/ingenieria/avionica
ls -ld /srv/aurora6/ingenieria/software
```

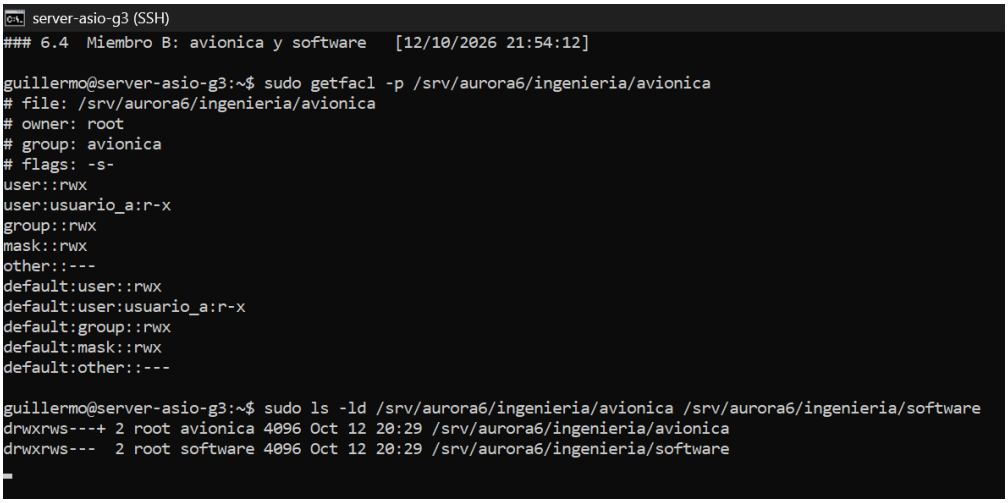


Ilustración 26. ACL de avionica y permisos de software en el servidor (miembro B)

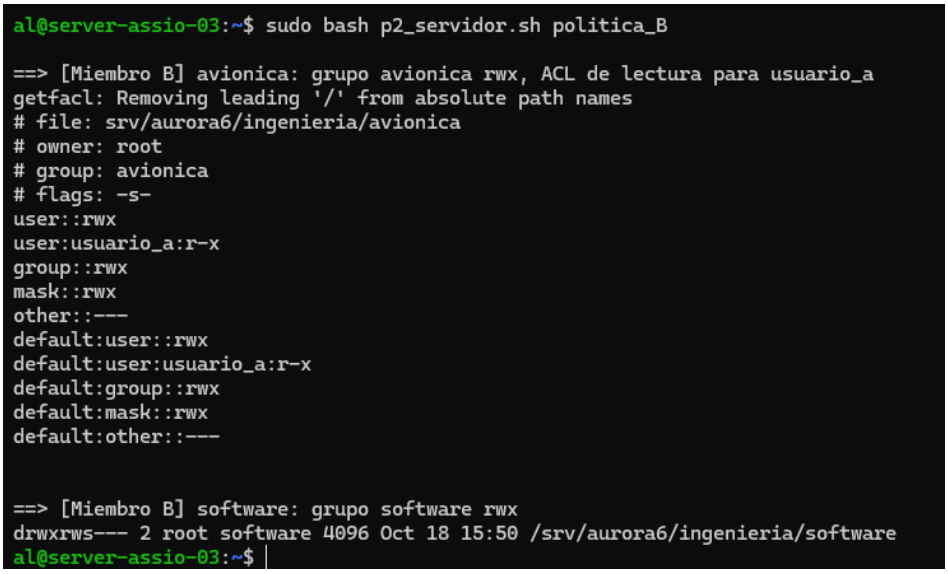


Ilustración 27. Política del miembro B aplicada por Alonso en su VM

Miembro C, Ariadna: `compras`, `documentacion/comun` y ACL de `contabilidad/facturas`.

```
sudo chown -R root:compras /srv/aurora6/compras
sudo chmod 2770 /srv/aurora6/compras /srv/aurora6/compras/proveedores
/srv/aurora6/compras/pedidos
sudo chown root:proyecto /srv/aurora6/documentacion/comun
sudo chmod 2750 /srv/aurora6/documentacion/comun
sudo chown root:root /srv/aurora6/contabilidad /srv/aurora6/contabilidad/facturas
sudo chmod 750 /srv/aurora6/contabilidad /srv/aurora6/contabilidad/facturas
sudo setfacl -m u:ariadna:--x /srv/aurora6/contabilidad
sudo setfacl -m u:ariadna:r-x /srv/aurora6/contabilidad/facturas
sudo setfacl -d -m u::rwx,g::r-x,o::---,u:ariadna:r-x /srv/aurora6/contabilidad/facturas
getfacl /srv/aurora6/contabilidad /srv/aurora6/contabilidad/facturas
```

El `chown -R` se limita a `/srv/aurora6/compras`, que en ese momento solo contiene sus dos subdirectorios vacíos, así que no afecta a otros datos. `documentacion/comun` queda con 2750 para que todo el proyecto pueda leer pero no escribir.

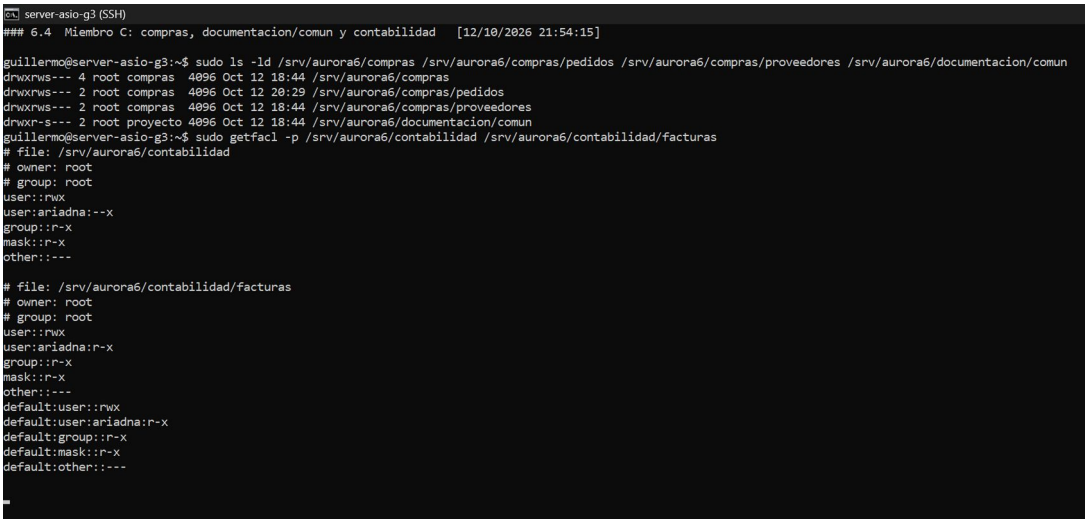


Ilustración 28. compras, documentacion/comun y ACL de contabilidad en el servidor (miembro C)

```
adminasv@server-asio-g3:~$ sudo bash p2.sh politicaC

==> [Miembro C] compras: grupo compras rwx (incluye proveedores y pedidos)
+ chown -R root:compras /srv/aurora6/compras
+ chmod 2770 /srv/aurora6/compras /srv/aurora6/compras/proveedores /srv/aurora6/compras/pedidos
+ ls -ld /srv/aurora6/compras /srv/aurora6/compras/pedidos /srv/aurora6/compras/proveedores
drwxrws--- 4 root compras 4096 Oct 19 11:11 /srv/aurora6/compras
drwxrws--- 2 root compras 4096 Oct 19 11:11 /srv/aurora6/compras/pedidos
drwxrws--- 2 root compras 4096 Oct 19 11:11 /srv/aurora6/compras/proveedores

==> [Miembro C] documentacion/comun: lectura para el grupo proyecto
+ set +x
+ chown root:proyecto /srv/aurora6/documentacion/comun
+ chmod 2750 /srv/aurora6/documentacion/comun
+ ls -ld /srv/aurora6/documentacion/comun
drwxr-s--- 2 root proyecto 4096 Oct 19 11:11 /srv/aurora6/documentacion/comun
+ set +x

==> [Miembro C] contabilidad: cerrada; ACL de paso para ariadna y de lectura en facturas
+ chown root:root /srv/aurora6/contabilidad /srv/aurora6/contabilidad/facturas
+ chmod 750 /srv/aurora6/contabilidad /srv/aurora6/contabilidad/facturas
+ setfacl -m u:ariadna:--x /srv/aurora6/contabilidad
+ setfacl -m u:ariadna:r-x /srv/aurora6/contabilidad/facturas
+ setfacl -d -m u::rwx,g::r-x,o::---,u:ariadna:r-x /srv/aurora6/contabilidad/facturas
+ getfacl /srv/aurora6/contabilidad /srv/aurora6/contabilidad/facturas
getfacl: Removing leading '/' from absolute path names
# file: srv/aurora6/contabilidad
# owner: root
# group: root
user::rwx
user:ariadna:--x
group::r-x
mask::r-x
other::---

# file: srv/aurora6/contabilidad/facturas
# owner: root
# group: root
user::rwx
user:ariadna:r-x
group::r-x
mask::r-x
other::---
default:user::rwx
default:user:ariadna:r-x
default:group::r-x
default:mask::r-x
default:other::---

adminasv@server-asio-g3:~$
```

Ilustración 29. Política del miembro C aplicada por Ariadna en su VM

6.5. Archivos de ejemplo para las pruebas

Para que las pruebas de lectura no dependan del orden en que trabaje cada uno, la cuenta de administración crea un archivo de ejemplo en cada recurso que se va a leer:

```
echo "Plan de ensayos estructurales AURORA-6" | sudo tee
/srv/aurora6/ingenieria/mecanica/plan_ensayos.txt
echo "Especificación de telemetría AURORA-6" | sudo tee
/srv/aurora6/ingenieria/avionica/telemetria.txt
echo "Normas comunes del proyecto AURORA-6" | sudo tee
/srv/aurora6/documentacion/comun/normas_proyecto.txt
echo "Factura 0001" | sudo tee /srv/aurora6/contabilidad/facturas/factura_0001.txt
sudo chmod 640 /srv/aurora6/documentacion/comun/normas_proyecto.txt
```

Usamos `sudo tee` porque `sudo echo ... > archivo` no funcionaría: la redirección la hace el shell del usuario sin privilegios antes de ejecutar `sudo`. En los directorios con ACL por defecto, cada archivo nuevo hereda sus entradas (por ejemplo `user:alonso` en mecánica) y la máscara se queda en `rw-`, porque los archivos se crean sin bit de ejecución.

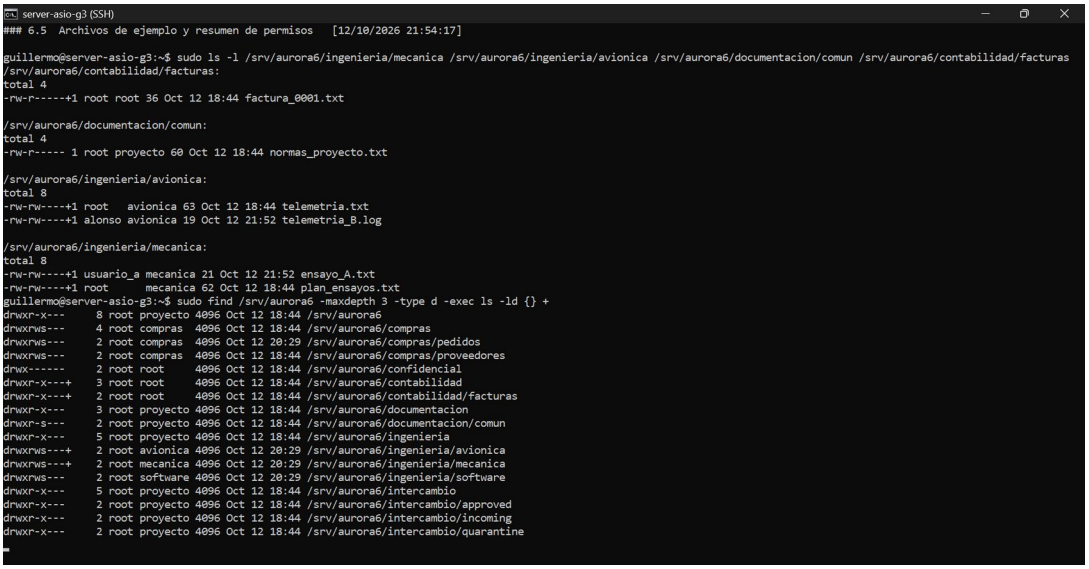


Ilustración 30. Archivos de ejemplo y resumen de propietarios, permisos y ACL de /srv/aurora6 en el servidor

7. Validación individual de la política de permisos

Cada prueba se ejecuta con la identidad del integrante correspondiente, y lo primero es comprobar con `whoami` e `id` que la sesión es de su usuario y ya incluye sus grupos. En el servidor, las pruebas se lanzaron el 12/10/2026 desde la sesión SSH de administración ejecutando cada orden con la cuenta de cada integrante mediante `sudo -u <cuenta>`; el núcleo aplica las mismas comprobaciones que en una sesión de ese usuario, porque el proceso corre con su UID y sus grupos, como confirma la salida de `id`. La cabecera de cada captura indica la sección, la cuenta usada y la hora del servidor.

Además, Alonso y Ariadna repitieron las pruebas en su propia VM iniciando sesión con su cuenta de trabajo, sin sudo, y el 20/10/2026 entraron al servidor del grupo por SSH con esa misma cuenta (sección 1.4). En las tablas, el resultado esperado sale de la configuración de la sección 6 y el obtenido resume la salida real que se ve en las capturas.

7.1. Pruebas del miembro A: Guillermo (usuario_a)

Tabla 8. Pruebas de usuario_a en el servidor

Prueba	Comando	Esperado	Obtenido
Escritura permitida en su departamento	`echo "Ensayo de vibración" > …/mecanica/ensayo_A.txt`	Se crea con grupo `mecanica` y `+` por la ACL por defecto	`-rw-rw----+ usuario_a mecanica … ensayo_A.txt`. Coincide
Lectura permitida en departamento ajeno	`cat …/avionica/telemetria.txt`	Muestra el texto	«Especificación de telemetría AURORA-6». Coincide
Escritura que debe rechazarse	`touch …/avionica/intento_A.txt`	Permission denied (la ACL solo da `r-x`)	Permission denied. Coincide
Acceso a directorios prohibidos	`ls /srv/aurora6/contabilidad`, `cd /srv/aurora6/confidencial`, `ls …/compras/pedidos`	Permission denied	Permission denied en los tres. Coincide
Lectura de documentación común	`cat …/comun/normas_proyecto.txt`	Muestra el texto	«Normas comunes del proyecto AURORA-6». Coincide
ACL de su recurso	`getfacl …/ingenieria/mecanica`	`user:alonso:r-x` y `group::rwx`	Aparecen, junto con `mask::rwx`, `other::---` y las líneas `default:`. Coincide

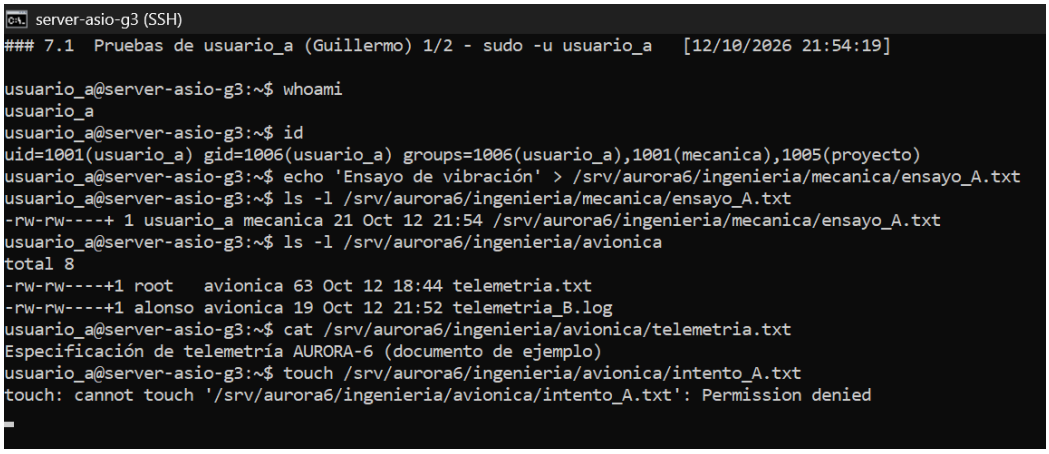


Ilustración 31. Pruebas de usuario_a en el servidor (1/2)

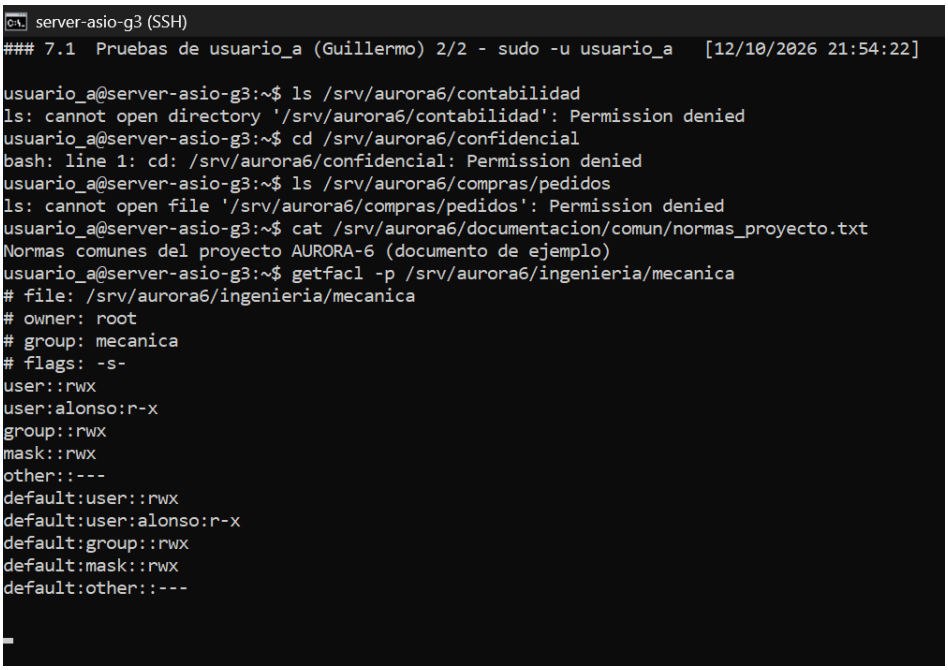


Ilustración 32. Pruebas de usuario_a en el servidor (2/2)

7.2. Pruebas del miembro B: Alonso (alonso)

Tabla 9. Pruebas de alonso

Prueba	Comando	Esperado	Obtenido
Escritura permitida en sus departamentos	`echo … > …/avionica/telemetria_B.log` y `echo … > …/software/control_B.py`	Se crean con grupo `avionica` y `software`	Se crean los dos con el grupo de cada directorio. Coincide
Lectura permitida en departamento ajeno	`cat …/mecanica/plan_ensayos.txt`	Muestra el texto	«Plan de ensayos estructurales AURORA-6». También lee `ensayo_A.txt` gracias a la ACL por defecto. Coincide
Escritura que debe rechazarse	`touch …/mecanica/intento_B.txt`	Permission denied	Permission denied. Coincide
Acceso a directorios prohibidos	`ls …/compras/pedidos`, `ls /srv/aurora6/confidencial`	Permission denied	Permission denied en ambos. Coincide
Lectura de documentación común	`cat …/comun/normas_proyecto.txt`	Muestra el texto	«Normas comunes del proyecto AURORA-6». Coincide
ACL de su recurso	`getfacl …/ingenieria/avionica`	`user:usuario_a:r-x` y `group::rwx`	Aparecen, con `mask::rwx` y las líneas `default:`. Coincide

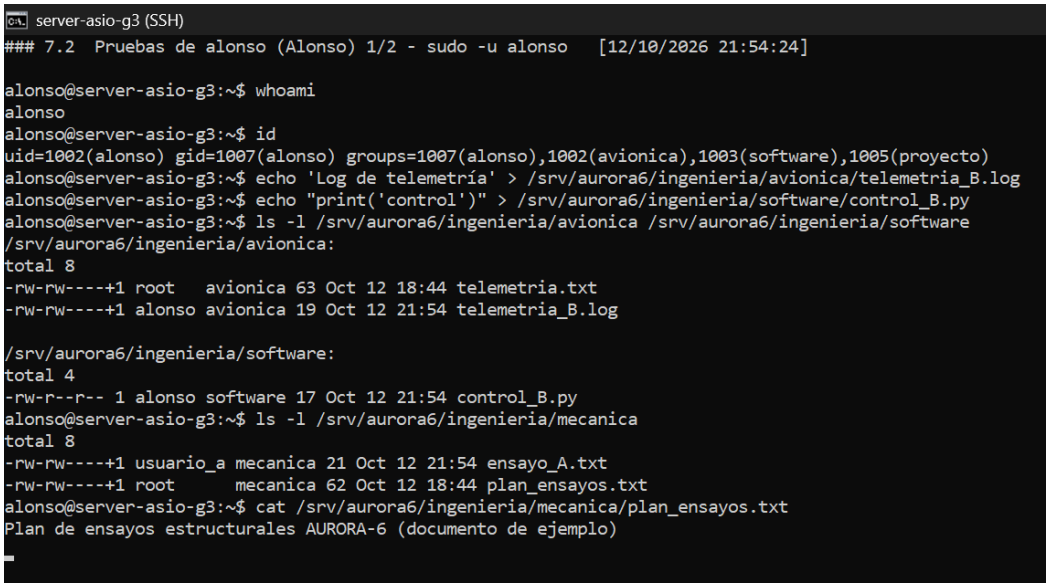


Ilustración 33. Pruebas de alonso en el servidor (1/2)

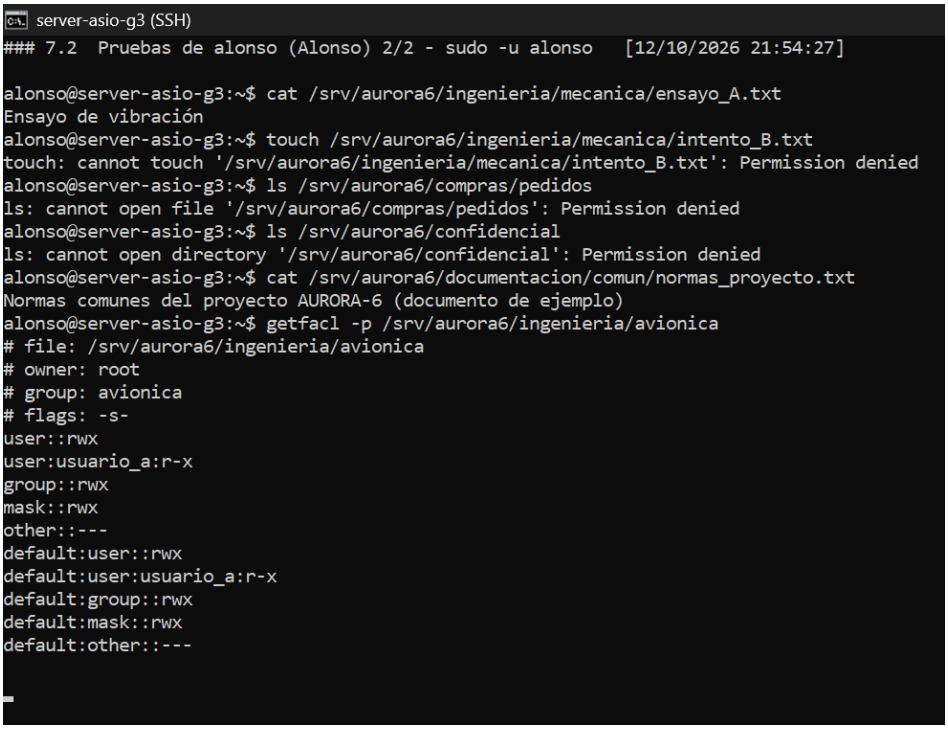


Ilustración 34. Pruebas de alonso en el servidor (2/2)

En su VM, Alonso inició sesión con la cuenta `alonso` y repitió la batería completa con el mismo resultado:

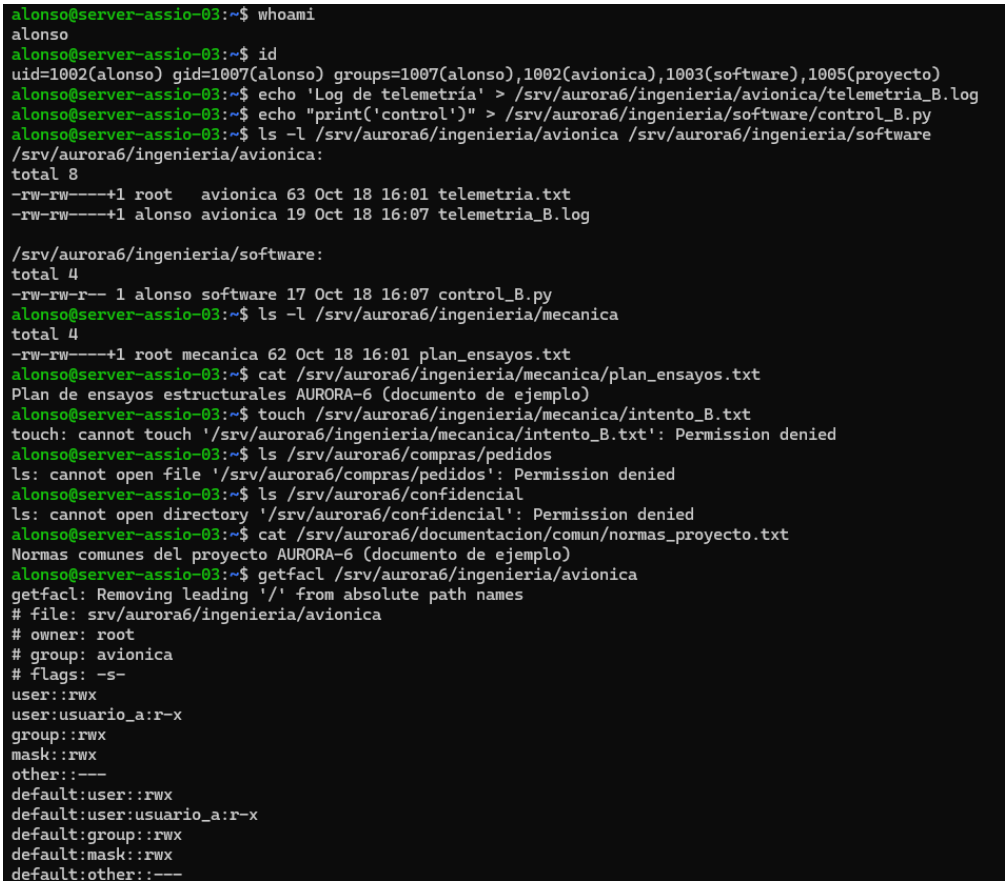


Ilustración 35. Pruebas de la política en la VM de Alonso con su cuenta de trabajo alonso

7.3. Pruebas del miembro C: Ariadna (ariadna)

Tabla 10. Pruebas de ariadna

Prueba	Comando	Esperado	Obtenido
Escritura permitida en su departamento	`echo … > …/compras/pedidos/pedido_001.txt` (servidor) y `touch …/pedidos/pedido001.txt` (VM)	Se crea con grupo `compras`	Se crea con propietario `ariadna` y grupo `compras`. Coincide
Lectura permitida en recurso ajeno	`cat …/facturas/factura_0001.txt` y `cat …/comun/normas_proyecto.txt`	Muestra el texto de ambos	«Factura 0001» y «Normas comunes del proyecto AURORA-6». Coincide
Paso sin listado en contabilidad	`ls /srv/aurora6/contabilidad`	Permission denied: solo tiene `--x`	Permission denied, aunque sí llega a facturas. Coincide
Escrituras que deben rechazarse	`touch …/mecanica/intento_C.txt` y `touch …/facturas/intento_C.txt`	Permission denied	Permission denied en ambos. Coincide
Acceso a directorio prohibido	`cd /srv/aurora6/confidencial`	Permission denied	Permission denied. Coincide
ACL de su recurso	`getfacl …/contabilidad/facturas`	`user:ariadna:r-x` y entradas `default:`	Aparecen, con `mask::r-x`. Coincide



Ilustración 36. Pruebas de ariadna en el servidor (1/2)

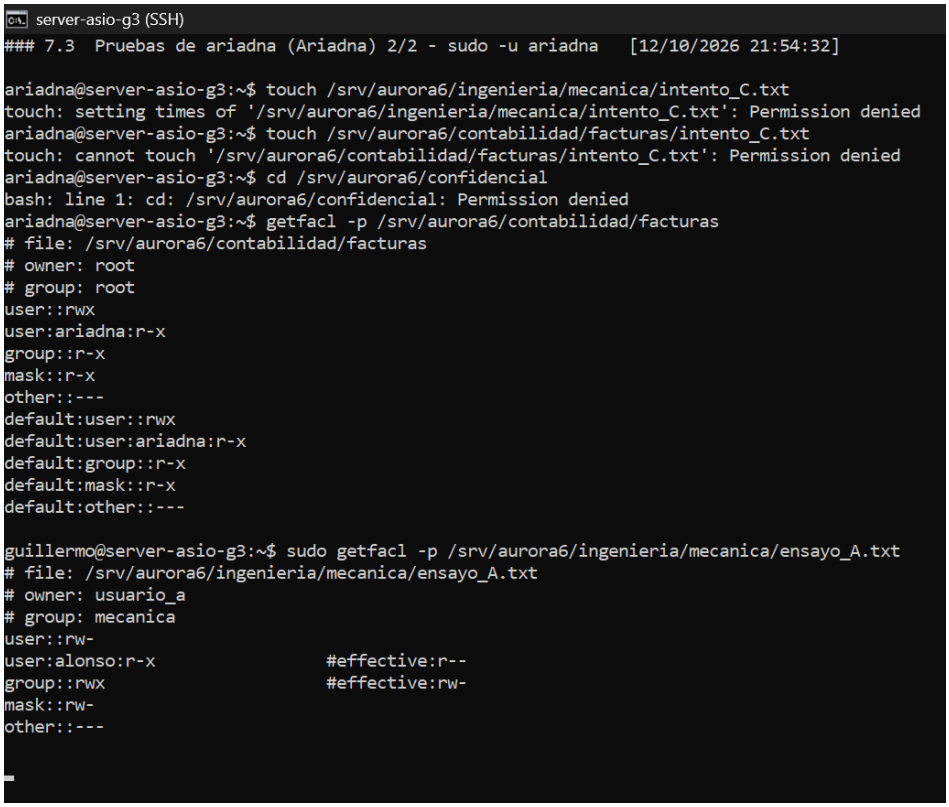


Ilustración 37. Pruebas de ariadna en el servidor (2/2) y herencia de la ACL por defecto en ensayo_A.txt

En su VM, Ariadna cerró la sesión de administración y entró con la cuenta `ariadna`, que ya no tiene sudo, para repetir las pruebas:

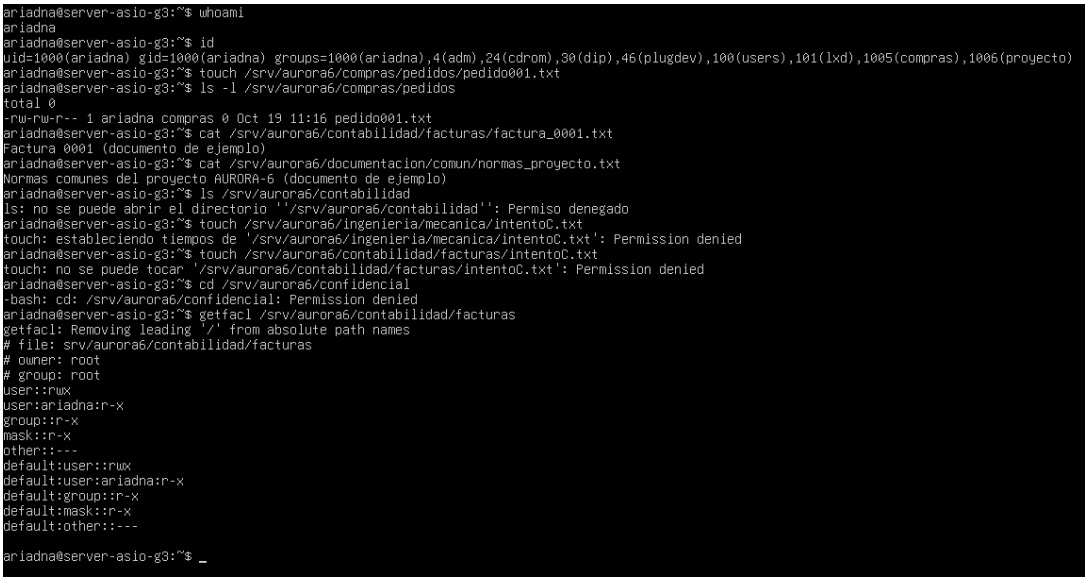


Ilustración 38. Pruebas de la política en la VM de Ariadna con su cuenta de trabajo ariadna

7.4. Interpretación

Las pruebas muestran el comportamiento real del sistema y no solo lo que dice `ls -l`. Las escrituras en el departamento propio funcionan porque el usuario está en el grupo propietario con `rwx`. Las lecturas cruzadas entre mecánica y aviónica funcionan gracias a la entrada de ACL `r-x`, y la escritura cruzada falla porque esa entrada no tiene `w`. Los accesos a confidencial,

contabilidad y pedidos fallan porque el usuario cae en la clase «otros», sin permisos, o porque le falta `x` en algún directorio de la ruta. En el caso de Ariadna, que `ls contabilidad` falle mientras `cat contabilidad/facturas/factura_0001.txt` funciona muestra muy bien la diferencia entre `r` y `x` en un directorio.

Todos los resultados coinciden con lo esperado, en el servidor y en las VMs. El `getfacl` de `ensayo_A.txt` confirma además la herencia de la ACL por defecto: el archivo que creó A en mecánica tiene la entrada `user:alonso:r-x`, limitada a `r--` por la máscara, y por eso Alonso puede leerlo. En la prueba de Ariadna sobre mecánica, `touch` dice «setting times of …» en lugar de «cannot touch», pero el resultado es el mismo: Permission denied y ningún archivo creado.

Por último, los archivos que crean Alonso en software y Ariadna en pedidos quedan con permisos 644 (`-rw-r--r--`), que es lo que produce la umask 022 por defecto en directorios sin ACL por defecto. No supone un riesgo, porque el 2770 del directorio impide el acceso a los otros, pero sí impide que los compañeros del mismo grupo modifiquen esos archivos; para las siguientes prácticas conviene usar una umask 007 o añadir también una ACL por defecto en software y compras.

8. Incidencias y soluciones

La tabla recoge los problemas reales que tuvimos durante la práctica, con su causa y cómo los resolvimos, y algunos fallos típicos que comprobamos que conviene tener presentes al aplicar esta configuración.

Tabla 11. Incidencias y soluciones

Síntoma	Causa	Solución
La VM del servidor se quedaba bloqueada al arrancar y el puerto 2222 rechazaba la conexión	La integridad de memoria y el hipervisor de Windows estaban activos, así que VirtualBox no podía usar VT-x	Desactivar la integridad de memoria y ejecutar `bcdedit /set hypervisorlaunchtype off` como administrador; se revierte con `auto`
La VM se cerraba sola al reanudarla desde un estado guardado	El estado se había guardado con la VM bloqueada en el arranque	Descartar el estado guardado y arrancar en frío
`ssh -p 2222 guillermo@localhost` fallaba con la VM encendida	En Windows `localhost` se resuelve primero a `::1` (IPv6) y la regla solo escuchaba en `127.0.0.1`	Conectar a `127.0.0.1` o dejar vacía la IP del anfitrión en la regla
Alonso y Ariadna no podían conectarse desde sus portátiles	La regla de reenvío estaba ligada a `127.0.0.1` y el firewall bloqueaba el puerto	Regla sin IP de anfitrión y regla de firewall para la subred local (sección 1.4)
Las cuentas de B y C se crearon como `usuario_b` y `usuario_c`	Después acordamos usar los nombres `alonso` y `ariadna`	`usermod -l alonso -d /home/alonso -m usuario_b` y `groupmod -n alonso usuario_b` (igual con ariadna); las ACL guardan el UID y siguen funcionando
En la VM de Ariadna la cuenta de trabajo tenía sudo	En la P1 `ariadna` era la cuenta de administración	Crear `adminasv` con sudo y quitar a `ariadna` del grupo `sudo`
`scp` no podía leer el script desde Descargas en el Mac	macOS no deja al Terminal acceder a Descargas sin permiso de privacidad	Copiar el script a la carpeta Público y enviarlo desde allí
Permission denied en el propio departamento	La sesión se abrió antes de añadir al usuario al grupo	Cerrar sesión y volver a entrar; `id` debe mostrar el grupo
La ACL aparece en `getfacl` pero no tiene efecto	Un `chmod` posterior redujo la máscara	`setfacl -m m::rwx ruta` o repetir `chmod` y `setfacl` en ese orden
B lista mecánica pero no puede leer un archivo nuevo de A	Falta la ACL por defecto	`setfacl -d -m …` y corregir los archivos existentes con `setfacl -m u:alonso:r--`

La incidencia de la cuenta de Ariadna es la que más afectaba a la propia política: si su cuenta de trabajo hubiera seguido en el grupo `sudo`, habría podido entrar en confidencial o leer toda la contabilidad con un simple `sudo`, aunque los permisos estuvieran bien. Separar la cuenta de administración de la de trabajo es lo que hace que la política se cumpla de verdad. Por la misma razón, conviene quitar también a `ariadna` del grupo `lxd`, que Ubuntu da al primer usuario y que permite obtener privilegios de root a través de contenedores: `sudo gpasswd -d ariadna lxd`.

9. Conclusiones

La práctica nos ha obligado a pasar de una política escrita en lenguaje de empresa a una configuración concreta del sistema de archivos. El paso más útil fue construir primero la matriz de accesos por usuario y directorio, incluidos los niveles intermedios, porque deja ver qué requisitos se cubren con grupos y cuáles no.

Los grupos de departamento resuelven la mayoría de los casos y son lo más fácil de mantener: dar de alta a un nuevo empleado consiste en añadirlo a un grupo. Las ACL solo han hecho falta en tres puntos, cuando un usuario externo al grupo necesitaba un permiso distinto sin abrir el recurso a todos (B en mecánica, A en aviónica y C en facturas). También hemos visto que una ACL de acceso sin ACL por defecto se queda corta para los archivos nuevos, y que la máscara puede anular una ACL sin que se note a simple vista, por lo que `getfacl` es imprescindible para comprobar.

Nos ha quedado clara la diferencia entre los permisos de un directorio y los de un archivo, sobre todo el papel de `x` como permiso de paso, que hemos aprovechado para que Ariadna llegue a las facturas sin ver el resto de la contabilidad. Montar el servidor compartido y conectarnos desde nuestros portátiles nos ha acercado a cómo se trabaja en un sistema real, y mantener las cuentas de trabajo sin sudo es lo que da sentido a toda la política: confidencial solo es confidencial si ninguna cuenta de trabajo puede elevar privilegios.

10. Referencias

[1] Sampedro Lirio, D. Administración de Servicios de Sistemas Operativos (88020). Enunciados de las prácticas P1 y P2. Universidad Camilo José Cela.

[2] Canonical. Ubuntu Server documentation. https://documentation.ubuntu.com/server/

[3] Linux man-pages. chmod(1), chown(1), useradd(8), usermod(8), groupadd(8), gpasswd(1). https://man7.org/linux/man-pages/

[4] Linux man-pages. acl(5), setfacl(1), getfacl(1). https://man7.org/linux/man-pages/

[5] Linux Foundation. Filesystem Hierarchy Standard 3.0. https://refspecs.linuxfoundation.org/fhs.shtml

[6] Microsoft. New-NetFirewallRule. https://learn.microsoft.com/powershell/module/netsecurity/new-netfirewallrule

[7] Oracle. VirtualBox User Manual: Port forwarding with NAT. https://www.virtualbox.org/manual/

Anexo A. Script de configuración del servidor

El script `p2_servidor.sh` reúne los comandos de las secciones 3 a 6 en bloques independientes, para que cada integrante pueda ejecutar su parte (`sudo bash p2_servidor.sh politica_A`) o para reconstruir el servidor completo (`sudo bash p2_servidor.sh todo`). Se puede ejecutar varias veces sin duplicar usuarios ni grupos, y `set -euo pipefail` lo detiene ante el primer error. Las contraseñas no se incluyen: se asignan de forma interactiva con `passwd`. En la VM de Ariadna se usó una copia con los bloques de política llamados `politicaA`, `politicaB` y `politicaC` y la traza de comandos activada, para que en las capturas se vieran las órdenes.

```
#!/bin/bash
# ASSIO (88020) - Práctica 2: estructura, usuarios, grupos, DAC y ACLs
# Proyecto AURORA-6. Se ejecuta en el servidor Ubuntu con la cuenta de administración:
#   sudo bash p2_servidor.sh <bloque>
# Bloques: estructura | usuarios | dac751 | base | politica_A | politica_B | politica_C |
semillas | verificar | todo

set -euo pipefail

# Nombres de las cuentas de trabajo (cámbialos por los acordados en el grupo)
USER_A="usuario_a"   # Miembro A: Ingeniería mecánica
USER_B="alonso"      # Miembro B: Aviónica y software
USER_C="ariadna"     # Miembro C: Compras y configuración

BASE="/srv/aurora6"

if [ "$(id -u)" -ne 0 ]; then
    echo "Ejecuta el script con sudo: sudo bash $0 <bloque>"
    exit 1
fi

for cmd in setfacl getfacl; do
    if ! command -v "$cmd" >/dev/null 2>&1; then
        echo "Falta $cmd. Instálalo con: sudo apt install acl"
        exit 1
    fi
done

paso() { echo; echo "==> $*"; }

estructura() {
    paso "Creando la estructura de $BASE"
    mkdir -p "$BASE"/ingenieria/{mecanica,avionica,software} \
             "$BASE"/compras/{proveedores,pedidos} \
             "$BASE"/contabilidad/facturas \
             "$BASE"/documentacion/comun \
             "$BASE"/confidencial \
             "$BASE"/intercambio/{incoming,approved,quarantine}
    tree -d "$BASE" 2>/dev/null || find "$BASE" -type d | sort
}

usuarios() {
    paso "Creando grupos de departamento y grupo común"
    for g in mecanica avionica software compras proyecto; do
        getent group "$g" >/dev/null || groupadd "$g"
    done

    paso "Creando cuentas de trabajo (sin privilegios sudo)"
    for u in "$USER_A" "$USER_B" "$USER_C"; do
        id "$u" >/dev/null 2>&1 || useradd -m -s /bin/bash "$u"
    done

    usermod -aG mecanica,proyecto          "$USER_A"
    usermod -aG avionica,software,proyecto "$USER_B"
    usermod -aG compras,proyecto           "$USER_C"

    for u in "$USER_A" "$USER_B" "$USER_C"; do
        id "$u"
        groups "$u"
    done
    echo
    echo "Asigna ahora la contraseña de cada cuenta de forma interactiva:"
    echo "  sudo passwd $USER_A ; sudo passwd $USER_B ; sudo passwd $USER_C"
}

dac751() {
    paso "Ejercicio 751 sobre documentacion"
    chown root:proyecto "$BASE/documentacion"
    chmod 751 "$BASE/documentacion"
    ls -ld "$BASE/documentacion"
```

```
}

base() {
    paso "Permisos base de los niveles superiores"
    chown root:proyecto "$BASE" "$BASE/ingenieria" "$BASE/documentacion" "$BASE/intercambio"
    chmod 750           "$BASE" "$BASE/ingenieria" "$BASE/documentacion" "$BASE/intercambio"
    chown root:proyecto "$BASE"/intercambio/{incoming,approved,quarantine}
    chmod 750           "$BASE"/intercambio/{incoming,approved,quarantine}

    paso "confidencial: solo root (gestión mediante sudo)"
    chown root:root "$BASE/confidencial"
    chmod 700 "$BASE/confidencial"
    ls -ld "$BASE" "$BASE"/* "$BASE/confidencial"
}

politica_A() {
    paso "[Miembro A] mecanica: grupo mecanica rwx, ACL de lectura para $USER_B"
    chown root:mecanica "$BASE/ingenieria/mecanica"
    chmod 2770 "$BASE/ingenieria/mecanica"
    setfacl -m u:"$USER_B":r-x "$BASE/ingenieria/mecanica"
    setfacl -d -m u::rwx,g::rwx,o::---,u:"$USER_B":r-x "$BASE/ingenieria/mecanica"
    getfacl "$BASE/ingenieria/mecanica"
}

politica_B() {
    paso "[Miembro B] avionica: grupo avionica rwx, ACL de lectura para $USER_A"
    chown root:avionica "$BASE/ingenieria/avionica"
    chmod 2770 "$BASE/ingenieria/avionica"
    setfacl -m u:"$USER_A":r-x "$BASE/ingenieria/avionica"
    setfacl -d -m u::rwx,g::rwx,o::---,u:"$USER_A":r-x "$BASE/ingenieria/avionica"
    getfacl "$BASE/ingenieria/avionica"

    paso "[Miembro B] software: grupo software rwx"
    chown root:software "$BASE/ingenieria/software"
    chmod 2770 "$BASE/ingenieria/software"
    ls -ld "$BASE/ingenieria/software"
}

politica_C() {
    paso "[Miembro C] compras: grupo compras rwx (incluye proveedores y pedidos)"
    chown -R root:compras "$BASE/compras"
    chmod 2770 "$BASE/compras" "$BASE/compras/proveedores" "$BASE/compras/pedidos"
    ls -ld "$BASE/compras" "$BASE"/compras/*

    paso "[Miembro C] documentacion/comun: lectura para el grupo proyecto"
    chown root:proyecto "$BASE/documentacion/comun"
    chmod 2750 "$BASE/documentacion/comun"
    ls -ld "$BASE/documentacion/comun"

    paso "[Miembro C] contabilidad: cerrada; ACL de paso para $USER_C y de lectura en
facturas"
    chown root:root "$BASE/contabilidad" "$BASE/contabilidad/facturas"
    chmod 750 "$BASE/contabilidad" "$BASE/contabilidad/facturas"
    setfacl -m u:"$USER_C":--x "$BASE/contabilidad"
    setfacl -m u:"$USER_C":r-x "$BASE/contabilidad/facturas"
    setfacl -d -m u::rwx,g::r-x,o::---,u:"$USER_C":r-x "$BASE/contabilidad/facturas"
    getfacl "$BASE/contabilidad" "$BASE/contabilidad/facturas"
}

semillas() {
    paso "Creando archivos de ejemplo para las pruebas de lectura"
    echo "Plan de ensayos estructurales AURORA-6 (documento de ejemplo)" >
"$BASE/ingenieria/mecanica/plan_ensayos.txt"
    echo "Especificación de telemetría AURORA-6 (documento de ejemplo)"  >
"$BASE/ingenieria/avionica/telemetria.txt"
    echo "Normas comunes del proyecto AURORA-6 (documento de ejemplo)"  >
"$BASE/documentacion/comun/normas_proyecto.txt"
    echo "Factura 0001 (documento de ejemplo)"                          >
"$BASE/contabilidad/facturas/factura_0001.txt"
    echo "Documento reservado (solo administración)"                    >
"$BASE/confidencial/reservado.txt"
```

```
    chmod 640 "$BASE/documentacion/comun/normas_proyecto.txt"
    chmod 600 "$BASE/confidencial/reservado.txt"
    ls -l "$BASE/ingenieria/mecanica" "$BASE/ingenieria/avionica" \
          "$BASE/documentacion/comun" "$BASE/contabilidad/facturas"
}

verificar() {
    paso "Resumen de propietarios, permisos y ACLs"
    find "$BASE" -maxdepth 3 -type d -exec ls -ld {} +
    echo
    getfacl -R -p "$BASE" 2>/dev/null | grep -v '^$' | grep -E '^# file|user:|group:|mask|
other' || true
}

case "${1:-}" in
    estructura) estructura ;;
    usuarios)   usuarios ;;
    dac751)     dac751 ;;
    base)       base ;;
    politica_A) politica_A ;;
    politica_B) politica_B ;;
    politica_C) politica_C ;;
    semillas)   semillas ;;
    verificar)  verificar ;;
    todo)       estructura; usuarios; dac751; base; politica_A; politica_B; politica_C;
semillas; verificar ;;
    *) echo "Uso: sudo bash $0 {estructura|usuarios|dac751|base|politica_A|politica_B|
politica_C|semillas|verificar|todo}"; exit 1 ;;
esac
```